**Preliminary Design Review**

**HEXAGON Flight**

**Team 03**




**Team Members**

Rylan Moseley: rylanmoseley@uky.edu

Griffin Liem: gali223@uky.edu

Angel Landaverde: anla240@uky.edu

Kathryn Chumbley: klch230@uky.edu

Ellie Carter: emca264@uky.edu

Bo Adams: baad236@uky.edu

**Professors**

Dr. Regina Hanneman: regina.hannemann@uky.edu

Dr. Robert Adams: rjadams@uky.edu

Dr. Sen-Ching Samson Cheung: sen-ching.cheung@uky.edu

**Teaching Assistants**

Sanzher Harsha: oshea.harsha@uky.edu

Santiago Posso Murillo: spo230@uky.edu

Anjani Priya Marthati: anjanipriya.marthati@uky.edu

**Project Team Information**

HEXAGON Flight is an undergraduate senior design team tasked with engineering a sub-250g fully autonomous indoor drone. Operating under the guidance of our project sponsor Dr. Robert Adams, our team is developing an edge-computed vision and autopilot architecture capable of real-time geometric shape recognition and autonomous obstacle traversal to demonstrate at the University of Kentucky E-Day event.

**Table of Contents**

List of Figures ..... iv

List of Tables ..... iv

1 Introduction ..... 1

1.1 Abstract ..... 1

1.2 Comments and Responses ..... 1

2 Problem Statement ..... 2

2.1 Objective ..... 2

2.2 Need ..... 2

2.3 Background ..... 2

3 Existing Devices ..... 3

3.1 User Story ..... 4

4 Requirements Specification ..... 5

4.1 Marketing Requirements ..... 5

4.2 Engineering Requirements ..... 6

4.3 Impact Statements ..... 12

5 Design ..... 14

5.1 Overall Design ..... 14

5.2 Behavioral Model ..... 14

5.3 Subsystem Designs ..... 18

5.4 Initial Overall Prototype ..... 31

6 Project Management ..... 31

6.1 Team Lead Information ..... 31

6.2 Work Breakdown Structure (WBS) ..... 34

6.3 Gantt Chart ..... 44

6.4 Costs ..... 49
7 Conclusion and Outlook on Future Work ..... 51
8 References ..... 51
9 Appendix ..... 52
9.1 Functional Decompositions ..... 52

## List of Figures

Figure 1. DJI Tello EDU Drone ..... 3
Figure 2. Amazon Prime Air Drone ..... 4
Figure 3. User Story ..... 4
Figure 4. High-level Behavioral Model ..... 15
Figure 5. Start-up Subroutines ..... 16
Figure 6. Takeoff and Background Subroutines ..... 17
Figure 7. Flight and Landing Subroutines ..... 18
Figure 8. ShaperDrone Drone Frame Design Drawing ..... 19
Figure 9. Drone Airframe Preliminary Drawing ..... 19
Figure 10. Flywoo GOKU GN745 AIO MicoAir MTF-02P optical flow sensor ..... 22
Figure 11. MTF-02P Optical Flow Sensor and VL53L0X Time-of-Flight sensor ..... 22
Figure 12. VL53L0X ToF Sensor ..... 23
Figure 13. Object moving at high-speed comparison [13] ..... 28
Figure 14. User Interface Design ..... 30
Figure 15. ShaperDrone Course Prototype ..... 30
Figure 16. Gantt Charts Outlining WBS Fall 2026 ..... 47
Figure 17. Gantt Charts Outlining Subsystem Integration Spring 2026 ..... 49
Figure 18. Level 0 Functional Decomposition ..... 52
Figure 19. Level 1 Functional Decomposition ..... 53
Figure 20. Power System Level 2 Functional Decomposition ..... 54
Figure 21. Sensors Level 2 Functional Decomposition ..... 55
Figure 22. Autopilot Level 2 Functional Decomposition ..... 56
Figure 23. Vision System Level 2 Functional Decomposition ..... 58
Figure 24. User Interface Level 2 Functional Decomposition ..... 59
Figure 25. Drone Course Level 2 Functional Decomposition ..... 60

## List of Tables

Table 1. Marketing Requirements ..... 5
Table 2. Engineering Requirements ..... 7
Table 3. Marketing to Engineering Requirement Traceability ..... 11
Table 4. Drone Weight Budget ..... 21
Table 5. System Power Budget ..... 22
Table 6. Single-Board Computer (SBC) Selection ..... 24
Table 7. Team Lead Summary ..... 32
Table 8. Work Breakdown Structure ..... 34
Table 9. Development Cost Breakdown ..... 50
Table 10. Rebuild Cost Breakdown ..... 50
Table 11. Power System IO Table ..... 54
Table 12. Sensor IO Table ..... 55
Table 13. Autopilot IO Table ..... 57
Table 14. Vision System IO Table ..... 58
Table 15. User Interface IO Table ..... 59
Table 16. Drone Course IO Table ..... 60

# 1 Introduction

## *1.1 Abstract*

HEXAGON Flight presents the Preliminary Design Review of an autonomous sub-250 g indoor quadcopter engineered to perform real-time shape recognition and precision gate traversal. Designed for the University of Kentucky Engineering Day showcase, the platform integrates an edge-computed vision pipeline running on a Radxa Zero 3W single-board computer with a low-level ArduPilot flight controller over high-speed MAVLink UART bridges. By employing an OV9281 global-shutter monochrome camera equipped with retroreflective illumination and optical flow odometry, the drone localizes and navigates through a user-specified sequence of geometric hoops (Circle, Triangle, Square) without relying on external motion-capture or GPS infrastructure. This report details the complete system architecture, including structural airframe design, power distribution, software finite state machine behavior, Artificial Potential Field (APF) trajectory planning, and rigorous system engineering requirements.

## *1.2 Comments and Responses*

The initial presentation of our design and requirements included no specification for the external safety equipment and enclosure required by the university. In response, we revised our Marketing and Engineering Requirements to formally incorporate the drone safety netting, defining physical perimeter dimensions, structural mesh constraints, and spectator safety standards, while linking these specifications directly to the Health and Safety Impact Statements. Similarly, the physical constraints and sequencing of the course goals were previously left unspecified. We resolved this by formalizing discrete gate geometry and defining measurable traversal tolerances for each shape in the sequence.

The Engineering Requirements originally presented were formatted in a manner that separated measurable numerical tolerances from the requirement descriptions, complicating empirical verification. To address this feedback, we restructured the matrix so that all requirements stand alone as complete normative clauses with integrated quantitative tolerances. Furthermore, we split and restructured our original ER 14, which combined target shape identification with global localization, into distinct standalone requirements designated as ER 14 and ER 15, providing each with an independent subject, testing protocol, and pass/fail metric.

Faculty concern was also raised regarding flight duration and battery life, noting that the original six-minute threshold was inadequate for a continuous multi-hour demonstration. Thermodynamic limits and strict mass constraints prevent a sub-250 g drone from sustaining a continuous sixty-minute airborne flight on a single pack. We addressed this feedback in ER 17 by explicitly decoupling single-flight endurance from cumulative demonstration availability. While maintaining a single-flight threshold of at least six minutes, we instituted an operational requirement for multiple on-hand flight packs alongside a rapid four-battery rotation and field-charging cycle to sustain continuous operation throughout the event.

Finally, our original presentation and background section lacked sufficient context regarding existing commercial solutions and prior art. We resolved this by expanding the background review to include a comparative benchmark against commercial off-the-shelf micro-UAV platforms, evaluating how their size, compute capabilities, and flight characteristics compare to our autonomous design. In addition, our functional decomposition has been revised to provide deeper architectural detail for the vision-based recognition, autonomous path planning, and flight control subsystems, establishing explicit behavioral models and signal flow across all operational states.

# 2 Problem Statement

## *2.1 Objective*

The objective of this object is to design a fully autonomous drone capable of identifying various shapes and flying through them for an E-day presentation at the University of Kentucky. The system aims to encourage incoming students to pursue engineering through the visual demonstration.

The primary objectives of the design are safety, reliability, and cost-effectiveness. The drone needs to be safe enough to pose zero risk to the spectators upon demonstration. The drone also needs to be reliable enough to succeed in 95% of its demonstration flights. Lastly, the drone needs to be cheap, with a weight limit of 250 grams, and a budget of $500, which ensures that the drone can be rebuilt by anybody for a relatively low cost.

## *2.2 Need*

E-Day is the University of Kentucky Stanley and Karen Pigman College of Engineering's annual open house, a free event where students, families, and anyone curious about engineering can explore roughly 150 demonstrations, exhibits, and contests presented by UK students, departments, government organizations, and industry. The college's dean has even noted that many UK engineering students had their first exposure to the field at E-Day. The autonomous drone senses its surroundings, makes decisions, and flies on its own, bringing together mechanical design, electrical systems, and software in a single system visitors can watch work. For young attendees, it shows how the math and science from the classroom become technology that moves through the real world, which could lead them towards engineering in the future. The drone also speaks to the industry partners and professional societies that support E-Day, since autonomous drones are already being used to inspect infrastructure, monitor crops, map job sites, and deliver packages. By presenting an autonomous drone at E-Day, we can inspire the next generation of Kentucky engineers while showing how student innovation connects to industry needs.

## *2.3 Background*

This project is a second attempt at a senior design project originally designed during the previous academic year. The previous team successfully completed and integrated most of the required subsystems. However, they were unable to achieve stable flight of the drone. Because of this, our team was given the option of either continuing development from the previous platform or beginning a completely new design. After evaluating the existing hardware, the team decided to reuse several components and resources from the previous project, including the flight-testing enclosure, flight controller, and selected sensors.

The existing flight testing area consists of a 36-square-foot enclosed space designed to provide a safe environment for indoor flight and development. Reusing portions of the previous system allows the team to reduce costs and development time while building upon the lessons learned from the previous design iteration.

Several components will be replaced to improve overall system performance and reliability. These include the single board computer (SBC), camera, motors, and air frame design. The team selected a new SBC with increased processing power and memory capacity to better support computer vision algorithms, sensor integration, and autonomous navigation tasks. By upgrading these components, the team aims to improve system responsiveness, reduce computational bottlenecks, and increase the likelihood of achieving stable autonomous flight.

The project must also comply with applicable regulations established by both the Federal Aviation Administration (FAA) and the University of Kentucky. While the FAA does not typically regulate drone operations conducted exclusively indoors, drones weighing more than 250 grams must generally be registered when operated outdoors. University regulations require all drones used on campus to be registered with the university and flown only during approved testing periods. These requirements must be considered throughout the design and testing process to ensure safe and compliant operation.

## 3 Existing Devices

Autonomous drone technology has seen significant development in recent years across commercial, industrial, and educational applications. Many existing drone platforms demonstrate autonomous navigation, obstacle avoidance, and environment sensing capabilities that are relevant to this project.

One example is the DJI Tello EDU [1], a programmable drone designed primarily for educational applications. The platform combines basic autonomous flight capabilities with programming interfaces that allow students to develop and test autonomous behaviors. The Tello EDU is commonly used in robotics competitions, computer science education, and engineering projects. Its ability to navigate predefined paths and respond to user-created control algorithms demonstrates the feasibility of compact autonomous drone systems.



Figure 1. DJI Tello EDU Drone

Another example is Amazon Prime Air [2], an autonomous drone delivery platform designed to transport small packages directly to customers. These drones utilize advanced sensing systems and navigation algorithms to detect obstacles, avoid hazards, and safely travel to delivery locations. The system can transport packages weighing up to five pounds while operating as a fully electric vehicle. Although the mission objectives differ from those of this project, the technologies used for obstacle detection, autonomous navigation, and flight stabilization are directly applicable.



Figure 2. Amazon Prime Air Drone

These examples demonstrate that autonomous drones can successfully operate in a wide range of environments and applications. The technologies utilized by these systems, including computer vision, obstacle avoidance, sensor integration, and autonomous flight control, provide valuable references for the development of the proposed indoor autonomous drone platform.

### *3.1 User Story*

A user story exists to illustrate how end users will utilize a product. Our User Story serves to showcase the drone's intended demonstration at the E-Day event at the University of Kentucky. The User Story is shown below in Figure 3.

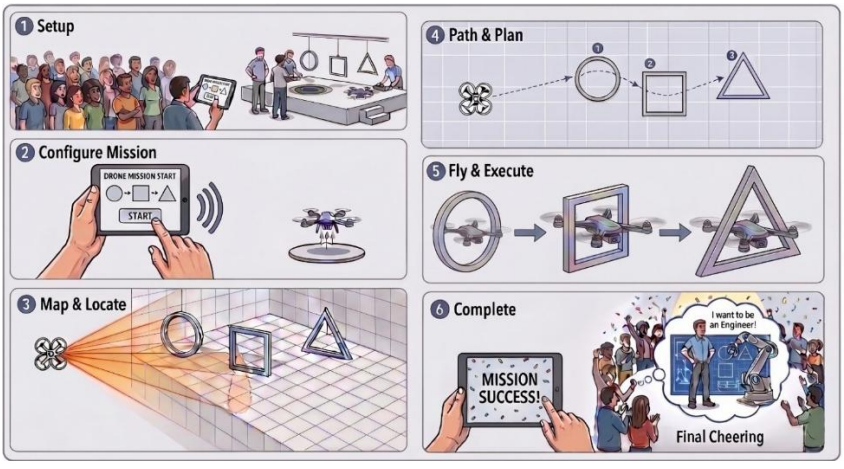



Figure 3. User Story

The User Story is described using six separate panels. The first panel shows the setup for the demonstration at E-Day. This will include setting up the net enclosure the drone will fly in, choosing positions for the shaped gates, and making sure the drone and its UI are in working order. The panel also shows the audience of prospective engineering students. The second panel is where the user inputs their desired hoop sequence. Panels three through five showcase the drone's autonomous capabilities: the vision system locates and maps the targets in panel three, the autopilot program plans the flight path in panel four, and the flight controller executes that path in panel five. The User Story ends in panel six with the drone successfully flying through the gates in order. The audience applauds and all collectively decide to come to the University of Kentucky to study engineering.

# 4 Requirements Specification

## *4.1 Marketing Requirements*

To ensure the drone meets expectations, we established a core set of marketing requirements. The table below lists these requirements to outline what the system must accomplish to deliver a successful E-Day demonstration.

Table 1. Marketing Requirements

| MR# | Marketing Requirement |
|---|---|
| 1 | The drone **shall** achieve stable, controlled flight in an enclosed indoor environment |
| 2 | The drone **shall** execute its entire sequence, from takeoff to landing, autonomously without manual intervention |
| 3 | The drone **shall** be lightweight (sub-250g) |
| 4 | The system **shall** minimize hazards to users or spectators |
| 5 | The drone **shall** detect, identify, and localize the three geometric goals |
| 6 | The drone **shall** navigate to and fly through each target goal in the exact sequence designated by the user |
| 7 | The drone **shall** have sufficient battery life to complete multiple demonstrations (takeoff, full sequence of goals, and landing) on a single charge |
| 8 | The system **should** provide an intuitive user interface that allows the operator to select and submit a sequence of target goals prior to flight |
| 9 | The drone **should** be within budget |
| 10 | The system **should** be easy and quick to setup and use |
| 11 | The system **should** operate without dependence on external tracking or processing. |

### *4.1.1 Marketing Requirements Justification*

The following list describes the justifications behind each marketing requirement:

MR 1: The drone must be capable of stable flight within the constraints of an enclosed indoor environment to fulfill its purpose for demonstration.

MR 2: A large focus of the demonstration is the drone's autonomy, so the drone must be able to complete the flight sequence without any human intervention.

MR 3: The 250g weight limit is a strict requirement to maintain compliance with university and regulatory thresholds for indoor drone operations.

MR 4: The drone will be flying near a live audience, making it imperative that any physical hazards to the spectators are avoided.
MR 5: To complete the desired flight sequence, the drone must first accurately detect, identify, and localize the targets to provide data for flight planning.
MR 6: To showcase the drone's precise path planning and flight capabilities, the drone must successfully navigate and fly through the targets in the exact sequence the user designated.
MR 7: The drone must have sufficient battery life to complete multiple demonstrations to maintain the flow of the event overall. This requirement imposes a balanced battery life with minimum battery weight.
MR 8: To ensure the demonstration is interactive and accessible, the drone must feature an intuitive user interface. It must be user-friendly for everyone, allowing anyone, regardless of technical background, to select and submit their desired sequence of goals.
MR 9: Staying within the allocated budget ensures the final drone design can be practically realized without exceeding financial constraints.
MR 10: A quick and easy to configure design will minimize setup time, allowing faster deployment and maximizing time available for active flights.
MR 11: To ensure flexibility where the demonstration can take place, the drone must rely entirely on onboard tracking and processing.

### *4.2 Engineering Requirements*

The engineering requirements for the HEXAGON Flight autonomous drone system translate the qualitative operational goals outlined in the Marketing Requirements (Section 3.1) into rigorous, verifiable, and quantitatively bounded technical metrics. Each requirement has been formulated to ensure that the resulting quadcopter can operate safely within an enclosed indoor flight volume while executing an autonomous navigation mission through geometric apertures.

#### *4.2.1 Requirement Classification: SHALL vs. SHOULD*

In accordance with system engineering standards, the requirements are partitioned into two priority tiers:

SHALL Statements (Normative Baseline): These represent non-negotiable functional criteria essential for achieving a functioning Minimum Viable Prototype (MVP). They derive directly from mandatory safety regulations (e.g., FAA Part 107 sub-250 g threshold), core operational constraints (autonomous takeoff, traversal, and landing), and fundamental flight stability. Failure to satisfy a SHALL statement renders the prototype inoperable or non-compliant.

SHOULD Statements (Design Enhancements): These represent performance optimizations and secondary features that elevate the system's operational readiness, efficiency, or maintainability beyond the baseline MVP (e.g., rapid battery cold-swap times, budget margins, extended telemetry range).

### *4.2.2 Engineering Requirements*

Table 2 presents the complete suite of twenty-four engineering requirements, detailing requirement statements, traceable marketing requirement linkages, numerical tolerances, technical justifications, and empirical verification procedures.

Table 2. Engineering Requirements

| ER# | Engineering Requirement | MR# | Justification | Verification/Testing |
|---|---|---|---|---|
| 1 | The drone shall maintain a stable hover with an altitude variation of <= +-10.0 cm and horizontal drift of <= +-15.0 cm over a continuous 30-second window. | 1, 4 | Stable, reliable flight required to complete task | Measure positional drift using downward optical flow/ToF telemetry logs against a ground grid over 30 s of unassisted hover. |
| 2 | The airframe and motor mounts shall maintain structural integrity with zero loosened fasteners or structural cracking following a 1.0 m free-fall impact onto foam padding. | 1, 4 | Repeated demonstrations require reliability and consistency | Perform a static drop test from a height of 1.0 m; complete a post-drop physical inspection of joints and frame rigidity. |
| 3 | The drone shall achieve a controlled horizontal translational speed of v_xy >= 0.60 m/s in indoor body-frame translational modes. | 1, 2, 6 | Timely completion of routine | Command a 2.0 m linear waypoint step in position control mode; calculate mean velocity from onboard telemetry logs. |
| 4 | The propulsion system shall produce a minimum static thrust-to-weight ratio (TWR) of >= 2.0:1 (>= 500 g total lift at 100% throttle). | 1, 2, 6 | Supports stable, safe, consistent flight | Measure static thrust using a calibrated multi-rotor dynamometer test stand across 0 to 100% throttle steps. |
| 5 | The physical Center of Gravity (CoG) shall align within <= +-3.0 mm of the geometric center of the | 1, 2 | Simplifies control parameters, pain point for past groups | Suspend the fully integrated, flight-ready airframe along |

| | | | | |
|---|---|---|---|---|
| | propulsion axes along X and Y. | | | orthogonal axes using a plumb-line balance fixture. |
| 6 | The drone shall execute an autonomous vertical landing with a descent speed of v_z <= 0.25 m/s and touchdown impact <= 1.5 G. | 1, 2, 10 | Safe and easy-to-use | Log downward rangefinder velocity and 3-axis accelerometer peak G-forces during automated touchdown sequences. |
| 7 | The system shall complete the autonomous 3-goal traversal sequence in < 180 s without human operator control intervention. | 1, 2, 4, 5, 6, 10 | Core project requirement, ease-of-use | Timed arena trial executing autonomous takeoff, 3-gate detection and traversal, and landing; record runtime via base station clock. |
| 8 | The total gross takeoff mass (GTOM) including airframe, avionics, battery, and propeller ducts shall not exceed 249.0 g. | 3, 4, 10 | Core project requirement, FAA Restriction, University compliance | Measure total mass of the flight-ready quadcopter with battery installed using a calibrated digital scale (+-0.1 g accuracy). |
| 9 | The flight software shall trigger an automatic failsafe landing or motor cutoff within <= 200 ms upon telemetry link loss or fatal error. | 2, 4 | Avoid danger from a rogue drone to spectators and itself | Simulate telemetry loss by terminating the base station transmission; log flight controller response timestamp via blackbox logs. |
| 10 | The operator UI shall provide an emergency software stop button that de-energizes all motors within <= 150 ms of engagement. | 4, 8, 10 | Need an emergency method to completely stop drone function | Transmit emergency cutoff command while logging motor PWM signals via logic analyzer; verify cut latency <= 150 ms. |
| 11 | The propeller ducts shall maintain a physical clearance of >= 2.0 mm from spinning blade tips under 5.0 N static side-load. | 1, 4, 10 | Blade contact with obstacles will destabilize flight | Apply a calibrated 5.0 N lateral force to outer duct rings; measure inner tip clearance using feeler gauges. |

| | | | | |
|---|---|---|---|---|
| 12 | All onboard wiring, battery leads, and signal harnesses shall remain contained within the airframe envelope with 0.0 mm exposed loops. | 1, 4 | Rotor/wire contact may damage equipment and destabilize flight | Visual and tactile inspection confirming all wire bundles are enclosed in internal routing channels or braided sleeving. |
| 13 | Propulsion motor bell and stator surface temperatures shall remain <= 60.0°C after 5.0 min of continuous hover. | 1, 4 | High temperature may damage motors and affect future performance | Measure motor bell temperatures immediately upon touchdown using an infrared thermal imaging camera. |
| 14 | The vision pipeline shall identify and classify geometric goal shapes (Circle, Triangle, Square) with >= 90.0% accuracy over 50 test frames. | 2, 5, 6, 11 | Drone must be able to identify targets and its location to them to meet project requirements | Process a standardized test dataset of 50 captured image frames under arena lighting; evaluate classification accuracy. |
| 15 | The vision system shall estimate the relative 3D coordinate position (x, y, z) of target gates with an error of <= +-5.0 cm at 1.5 m. | 2, 5, 6, 11 | Drone can't reach targets if motion disturbs target tracking information | Compare visual pose estimates against ground-truth laser distance measurements from the camera plane to the gate aperture. |
| 16 | The drone shall traverse the open aperture of each target gate maintaining >= 5.0 cm clearance between duct edges and the frame. | 1, 5, 6, 11 | Collision with gates may lead to drone crashing | Record video of gate crossings with high-speed camera; measure closest physical proximity between drone and gate frame. |
| 17 | The battery power system shall provide >= 6.0 min of continuous flight per pack and support >= 60 min cumulative demo availability. | 1, 7 | Inadequate battery life leads to the drone unable to complete its tasks | Timed hover test to 15% capacity cutoff; verify rapid 4-pack battery swapping and continuous field charging. |
| 18 | The wireless user interface shall acknowledge command inputs and display telemetry | 2, 5, 6, 8 | Failure to deliver goal sequence means the | Measure round-trip ping-pong packet timestamps between UI |

| | | | | |
|---|---|---|---|---|
| | updates with a round-trip latency <= 250 ms. | | drone cannot generate or execute a flight path | ground client and onboard companion computer. |
| 19 | The wireless communication link shall maintain reliable packet delivery (< 2% packet loss) at a line-of-sight range >= 10.0 m. | 2, 8 | Loss of connectivity can cause drone to lose flight path | Stream telemetry datagrams across the 10.0 m arena boundary through netting; calculate packet drop rate. |
| 20 | The total Bill of Materials (BoM) rebuild expenditure for the flight vehicle shall not exceed $400.00. | 2, 9 | Team budget allowance | Sum invoice component expenditures for all flight-critical hardware and structural airframe elements. |
| 21 | The battery mounting mechanism shall support a cold-swap turnaround time of <= 90 s from touchdown to flight-ready state. | 4, 10 | Swappable batteries and quick reboot allow for quicker testing and longer flight uptime | Time battery removal, fresh pack insertion, connector mating, and software reboot across five consecutive trials. |
| 22 | The state estimation system shall maintain indoor body-frame position drift <= +-5.0 cm/s in translation and <= +-1.0°/s in yaw. | 1, 4, 5, 6, 11 | Position tracking of the drone allows for superior 3D mapping | Measure optical flow dead-reckoning displacement over a 60-second flight path without active visual fiducial updates. |
| 23 | The companion computer CPU core temperature shall remain <= 75.0°C under full computational load (100% vision/autopilot utilization). | 1, 4 | Thermal throttling can lead to drone malfunction | Log internal core temperature sensors while running simultaneous 30 FPS vision acquisition and path planning routines. |
| 24 | The flight area enclosure shall provide an enclosed volume >= 6.0 ft x 6.0 ft x 6.0 ft with mesh spacing <= 1 inch. | 4 | Physical boundary containment prevents flyaway hazard to spectators and defines the active obstacle navigation envelope. | Measure cage frame dimensions with a tape measure; verify netting aperture size with calipers (<= 20 mm). |

### *4.2.3 Requirement Traceability*

To guarantee that all customer expectations and capstone project needs are satisfied, Table 3 maps each Marketing Requirement to its corresponding Engineering Requirements.

Table 3. Marketing to Engineering Requirement Traceability

| Marketing Requirement (MR #) | Summary Description | Satisfying Engineering Requirements (ER #) |
|---|---|---|
| **MR 1** | Achieve stable, controlled indoor flight | ER 1, ER 2, ER 3, ER 4, ER 5, ER 6, ER 11, ER 12, ER 13, ER 16, ER 17, ER 23 |
| **MR 2** | Fully autonomous takeoff-to-landing operation | ER 1, ER 3, ER 4, ER 5, ER 6, ER 7, ER 9, ER 14, ER 15, ER 18, ER 19, ER 20 |
| **MR 3** | Lightweight construction (sub-250 g) | ER 8 |
| **MR 4** | Minimize hazard to users and spectators | ER 1, ER 2, ER 3, ER 4, ER 5, ER 6, ER 7, ER 9, ER 10, ER 11, ER 12, ER 13, ER 21, ER 22, ER 23, ER 24 |
| **MR 5** | Detect, identify, and localize geometric goals | ER 7, ER 14, ER 15, ER 16, ER 18, ER 22 |
| **MR 6** | Navigate and traverse goals in designated order | ER 3, ER 4, ER 5, ER 6, ER 7, ER 14, ER 15, ER 16, ER 18 |
| **MR 7** | Sufficient battery life for demonstrations | ER 17, ER 21 |
| **MR 8** | Intuitive UI for sequence selection and monitoring | ER 10, ER 18, ER 19 |
| **MR 9** | Financial feasibility within budget | ER 20 |
| **MR 10** | Rapid setup and user-friendly operation | ER 6, ER 7, ER 8, ER 10, ER 11, ER 21, ER 24 |
| **MR 11** | Operation without external tracking systems | ER 1, ER 7, ER 14, ER 15, ER 16, ER 22 |

## *4.3 Impact Statements*

### *4.3.1 Standards*

This project will follow several relevant safety and design standards to ensure reliable operation during both the development and demonstration of the drone. To ensure this happens, the standards will act as a guide for keeping safety, battery handling, and flight-control reliability at the forefront of our minds. Following established standards is important because the project combines moving components, electrical systems, wireless communication, and autonomous navigation in an indoor environment surrounded by onlookers.

The team is considering the following standards:

- FAA 14 CFR Part 107: Establishes federal operating rules for small, unmanned aircraft systems, including pilot responsibilities, safe operation, and restrictions on flying over people. These rules are useful references for understanding responsible drone operation [3].

- ASTM F3178-24: Provides guidance for assessing operational risks associated with unmanned aircraft systems (UAS). This supports identifying potential hazards during indoor flights and planning appropriate safety measures [4].

- ASTM F2910-22: This standard establishes design, construction, and testing requirements for small, unmanned aircraft systems. It is relevant to our autonomous drone project because it provides guidance for evaluating the drone's physical design, structural integrity, and overall safety [5].

- ASTM F3002-22: Addresses command-and-control systems for small UAS, including communication links and control functions. This is relevant to communication between the drone, onboard computer, and user interface [6].

- IEC 62133-2:2017+A1:2021: Establishes safety requirements and testing for portable rechargeable lithium batteries, supporting safe battery handling and use [7].

### *4.3.2 Health and Safety*

Because our project culminates in a public demonstration at UK engineering's E-day before an audience of young children and adults, health and safety considerations served as primary drivers throughout our design process. To eliminate physical risks associated with autonomous flight, we implemented a multi-layered safety architecture incorporating both hardware safeguards and software protocol. Physical hazards are mitigated through propeller guards, fully contained electrical wiring, and thermal throttling to prevent contact injuries, burns, and electrical shorts. Operation during all flight tests and demonstrations will be restricted to an enclosed mesh flight net. On the control side, our autonomous flight system incorporates a dedicated software failsafe which will automatically trigger the drone descent sequence to prevent erratic behavior if sensor communications degrade or connection loss occurs. In addition, the user interface features a prominent emergency stop button, allowing operators to immediately execute a hard kill switch if necessary to protect bystanders.

A successful realization of our project delivers an engaging, educational engineering demonstration that highlights responsible innovation while upholding rigorous safety standards to protect operators, participants, and spectators alike.

### *4.3.3 Manufacturability*

Manufacturability describes how easily and cost-effectively a product can be produced using available manufacturing technology and standardized techniques. This consideration directly impacted our drone's design in several critical ways. First, because our aircraft is subject to a strict weight limit, each component must be fabricated or sourced with precise tolerances to guarantee consistent weight distribution across assemblies. Second, our tight budget constraints influenced us to select off-the-shelf parts that are simple to manufacture, ensuring they will remain widely available from multiple commercial vendors at competitive market prices. To streamline production, assembly, and upkeep, we implemented a modular component and battery architecture, utilizing standardized fasteners, robust frame rigidity, and simple mounting plates. This ensures that individual subsystems can be quickly swapped or repaired during testing without requiring custom tooling. In addition, our user interface has been designed for straightforward deployment, featuring an intuitive setup process that minimizes configuration time.

A successful realization of our project will provide an accessible, easily reproducible drone built at a reasonable price point, making it highly viable for broader replication, additional academic research, and extended consumer use.

### *4.3.4 Social and Cultural*

Our project will be presented at the University of Kentucky's Engineering Day (E-Day) where students, faculty, and visitors of a wide demographic can learn about the advancements of autonomous technology. By demonstrating a drone that identifies shapes and navigates through targets in real-time, we hope to make complex engineering concepts more understandable and engaging for a broad audience. To make the demonstration more interactive, we are developing a user interface (UI) that will display key features such as the selected flight sequence, camera feed, telemetry data, and mission status. The telemetry data will include battery life, the current target of the drone, altitude, and other real-time information about the drone's status. This will allow visitors to better understand what the drone is doing and how it operates. Making this information visible and easy to follow is especially important for younger audiences, as it helps break down complex concepts into simple, visual explanations. By combining a live demonstration with the user interface, we aim to encourage curiosity about STEM while showing how programming, electrical, and mechanical systems work together. E-day provides us with the opportunity to make engineering more approachable and potentially inspire students to explore STEM fields in the future.

### *4.3.5 Economic*

The economic category encompasses the financial constraints, cost efficiencies, and economic feasibility associated with our design choices. Operating within a strict total project budget of $500 directly impacted our engineering decisions by dictating a lightweight, highly cost-effective build. To maximize budget efficiency, we selected hardware components that offer adequate functional performance without unnecessary high-end features that would inflate costs. This balanced strategy allowed us to procure all essential drone components while preserving a necessary financial contingency margin for replacement parts during testing and adjustment. Furthermore, prioritizing a lightweight frame enables us to optimize power consumption, allowing us to select an efficient battery system that extends flight time per charge and reduces overall operational costs.

A successful realization of our project will provide a fully autonomous drone designed and deployed within a low-cost framework. By providing affordable, lightweight, and capable architecture, our work will offer a viable economic template for educational institutions, researchers, and potential customers looking for cost-effective, autonomous, aerial solutions.

# 5 Design

## *5.1 Overall Design*

The complete design aims to create a small autonomous drone weighing less than 250g capable of flying through square, triangle, and circle shapes in any order the user chooses. However, for this semester, the goal is to create a drone capable of stable hovering, which successfully integrates the airframe, power system, autopilot, and sensors. The user interface and vision system must also be independently functional.

The overall design will function with a 3D-printed custom airframe of carbon-fiber PETG filament, with dedicated mounting holes for the ESC/Flight controller, raspberry pi, camera, and motors. The airframe will also have detachable landing gear and propeller guards for safety. Once in the air, the camera of the drone will scan the area for the three distinct shapes through the single board computer. For consistency, the shapes will be marked with reflective tape for more effective camera scanning, and the borders of the course will have AprilTags for better 3D mapping of the space. The drone will then wait for the input through the user interface, which once the shape sequence is selected, the flight path will be generated. This flight path will then be executed by the motors through commands given by the ESC/Flight controller in conjunction with sensor information. The drone will also have a start and stop sequence, with a designated takeoff/landing area.

Flight tests will be conducted at least once a week to verify the drone's flight stability on hovering as well as in motion. Component specific testing will be discussed in the corresponding sections below.

## *5.2 Behavioral Model*

The operational execution of the HEXAGON Flight autonomous drone follows a deterministic state machine managed by the Autopilot and Single-Board Computer (SBC) subsystem.

### *5.2.1 Overall Behavioral Model*

At the highest level, the drone's operation proceeds through a structured lifecycle as outlined in Figure 4. This sequence begins with physical battery connection and power rail distribution during the power-up and boot phase, followed by sensor calibration, optical flow zeroing, serial MAVLink heartbeat synchronization, and local Unix socket binding during the initialization sequence. From there, the system commands automated arming, motor spool-up, vertical ascent, and stabilization at a 1.0 m holding altitude. Once stable, the drone executes a fixed-altitude yaw scan, acquires target fiducial markers via the vision pipeline, and computes trajectory waypoints using an Artificial Potential Field (APF) controller before finally approaching and passing through the three target geometric gates and returning for touchdown and disarm

### *5.2.2 Subroutine Decompositions*

Within those operational phases, key software subroutines govern the real-time execution of the autonomy stack as illustrated in Figure 5. The Autopilot Initialization Routine handles peripheral discovery, socket binding, UART baud negotiation at 921,600 baud, and initial MAVLink telemetry stream verification. Once initialized, the Autopilot Iteration Routine serves as the core 20 Hz periodic loop responsible for telemetry ingestion, state estimation updates, failsafe watchdog monitoring, and command dispatch. Finally, the Drone Pathfinding and Traversal Routine acts as the high-level navigation state machine that steps through the target shape sequence, plans waypoints, and adjusts trajectory commands

In addition to normal path execution, the behavioral model handles critical fault conditions via two distinct emergency stop protocols, which are governed by the background process loops shown in Figure 6. A "Hard" Stop instantly de-energizes all motor PWM outputs upon an operator manual trigger or critical hardware failure. Alternatively, a "Soft" Stop aborts the active traversal maneuver, halts forward velocity, and initiates an immediate controlled descent or return-to-land sequence.

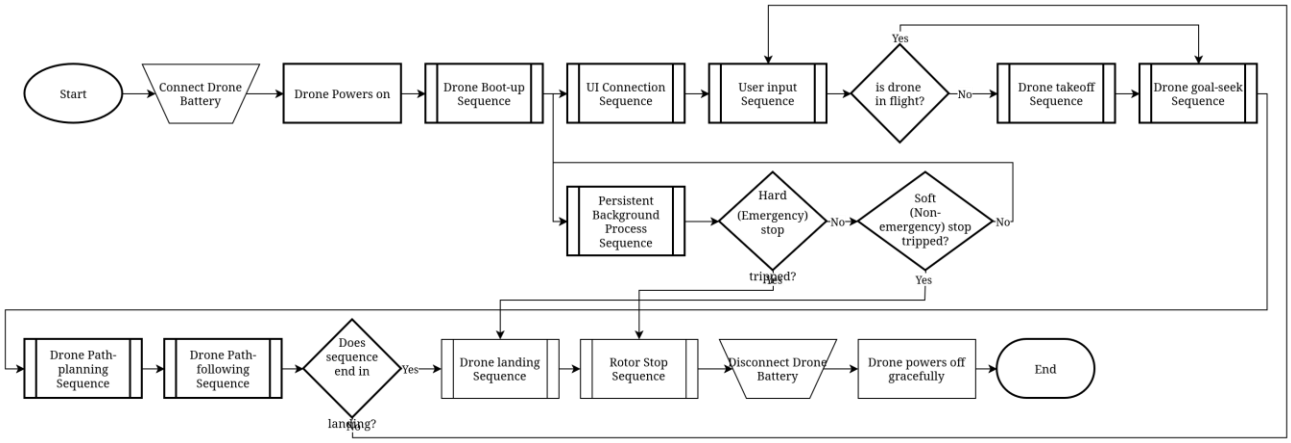



Figure 4. High-level Behavioral Model

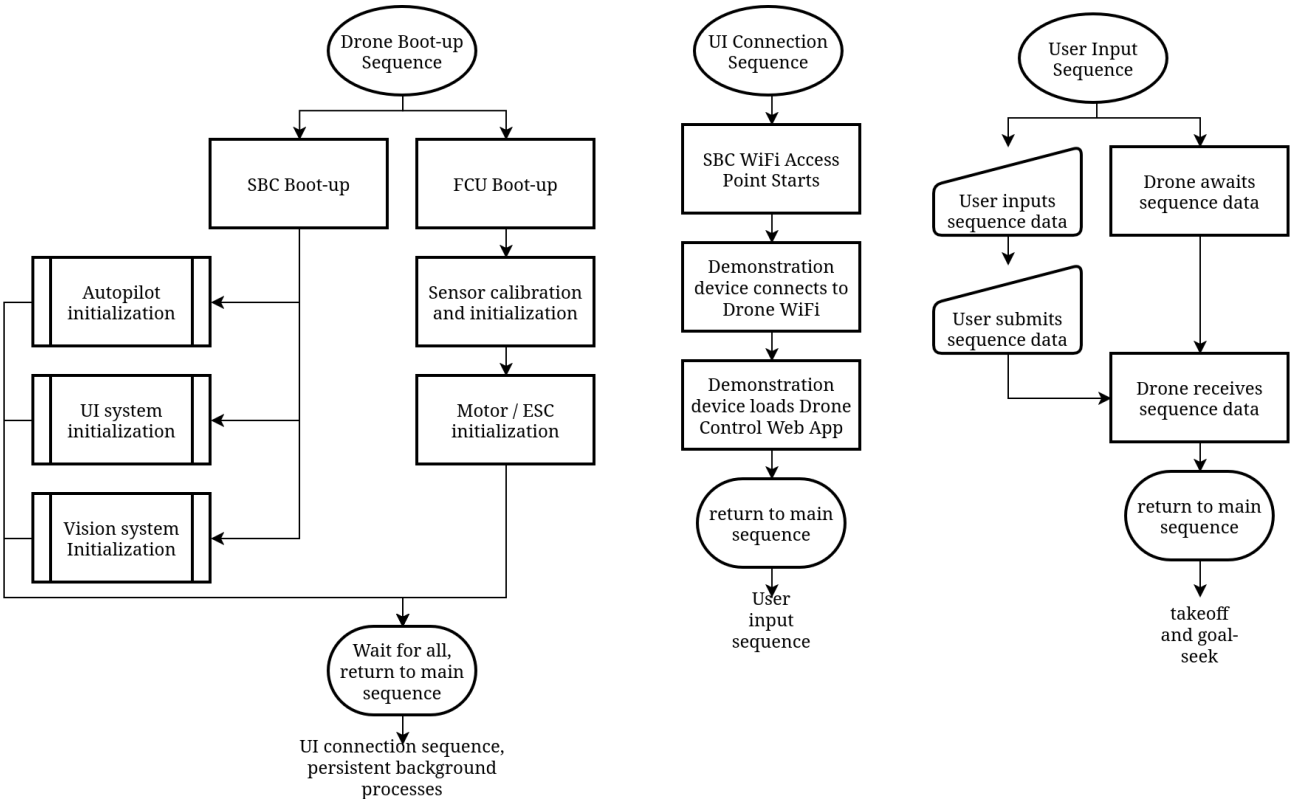



Figure 5. Start-up Subroutines

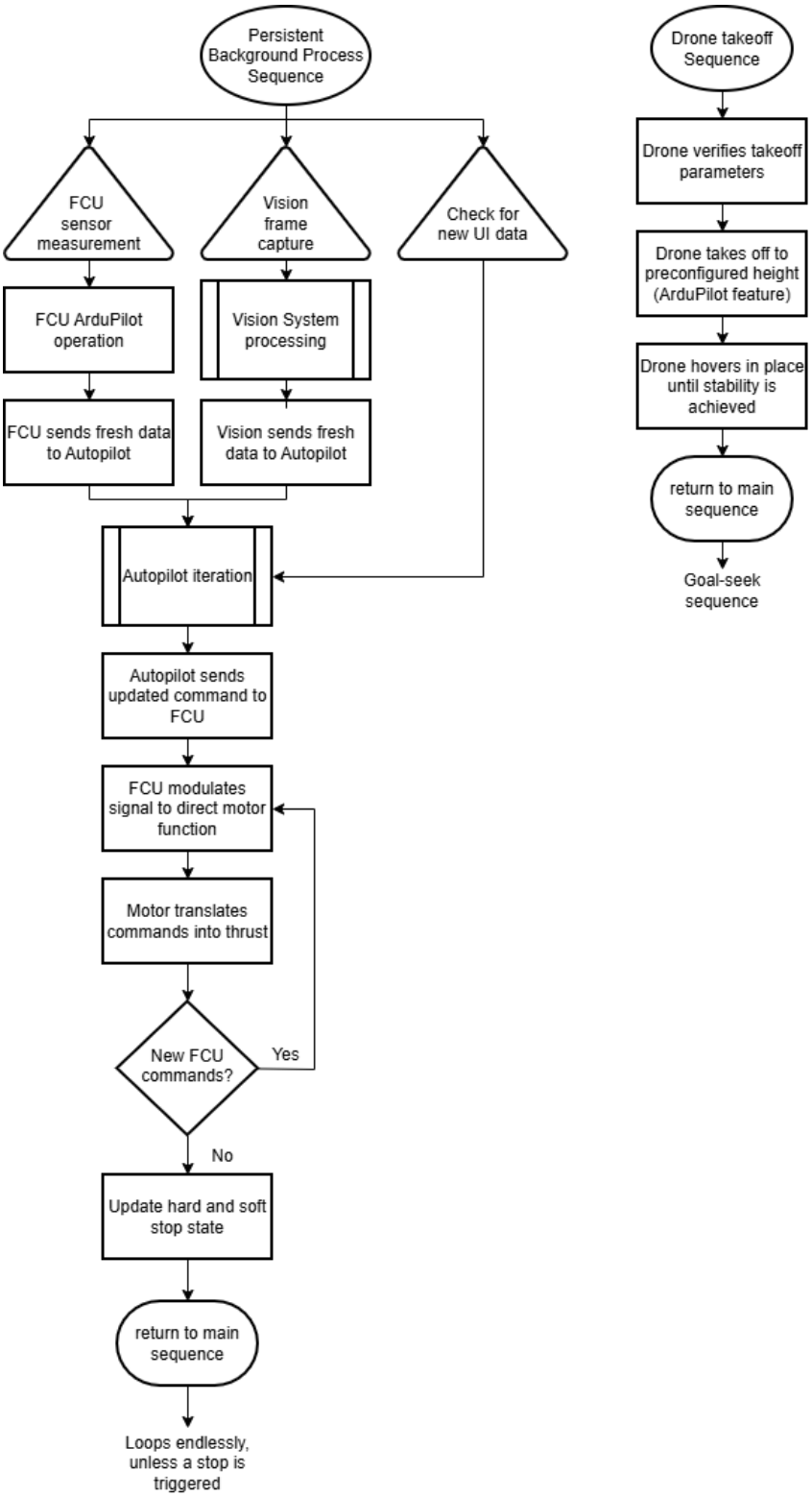



Figure 6. Takeoff and Background Subroutines

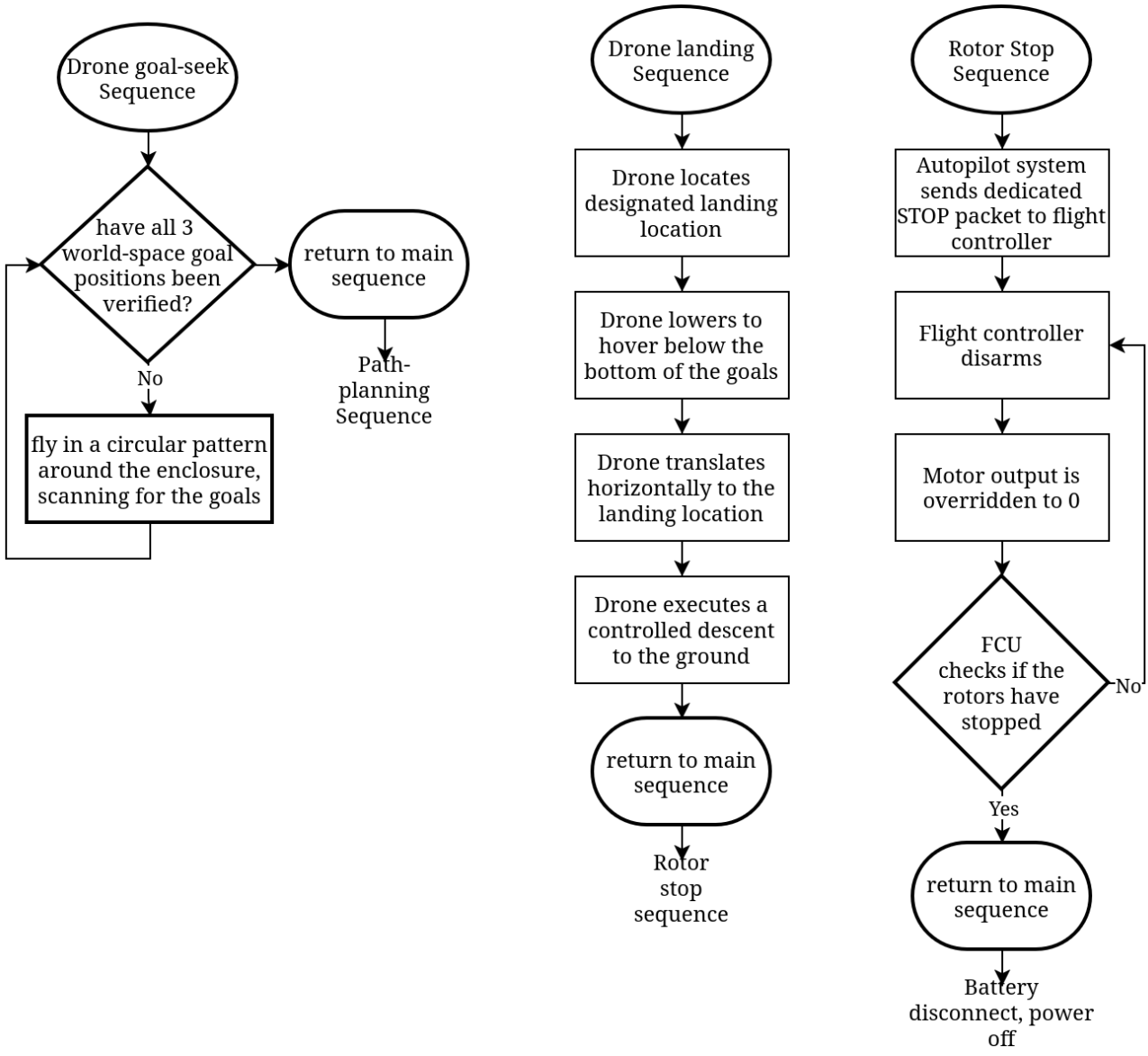



Figure 7. Flight and Landing Subroutines

## *5.3 Subsystem Designs*

### *5.3.1 Airframe*

The final airframe design will be complete, with a 3D-printed part of PLA filament available for mounting. The design takes inspiration from the previous team's design (seen in Figure 8), with supports connecting the drone arms that also mount the optical flow sensor and the time-of-flight sensor. The old design also is not optimized for 3D printing, with many overhangs present with many different parts that must be bolted together.

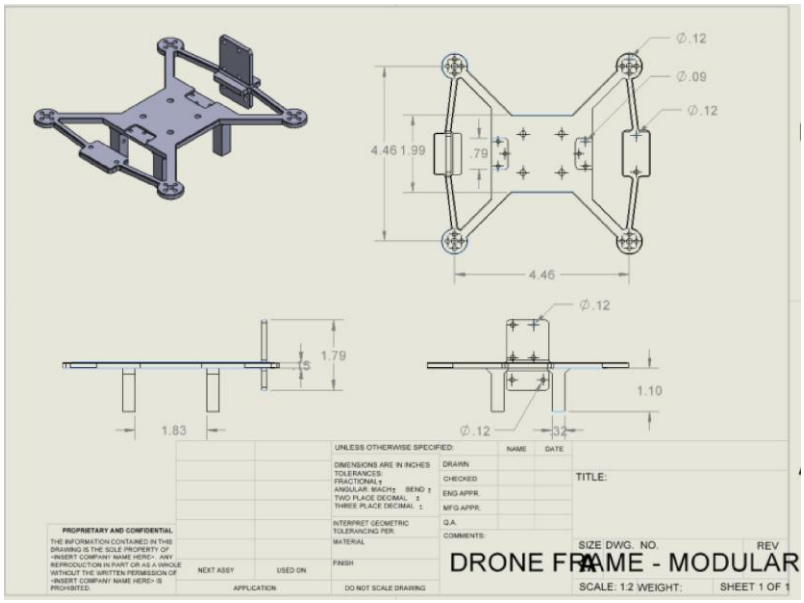



Figure 8. ShaperDrone Drone Frame Design Drawing

The current airframe design can be printed in one piece without supports and has mounting holes for all required parts apart from the sensors and camera. The landing gear will also be modular and connect to the corners of the drone by the motors as opposed to branching from the center of the frame like in the previous design. A drawing with important dimensions of the new drone frame design can be seen in Figure 9 below.

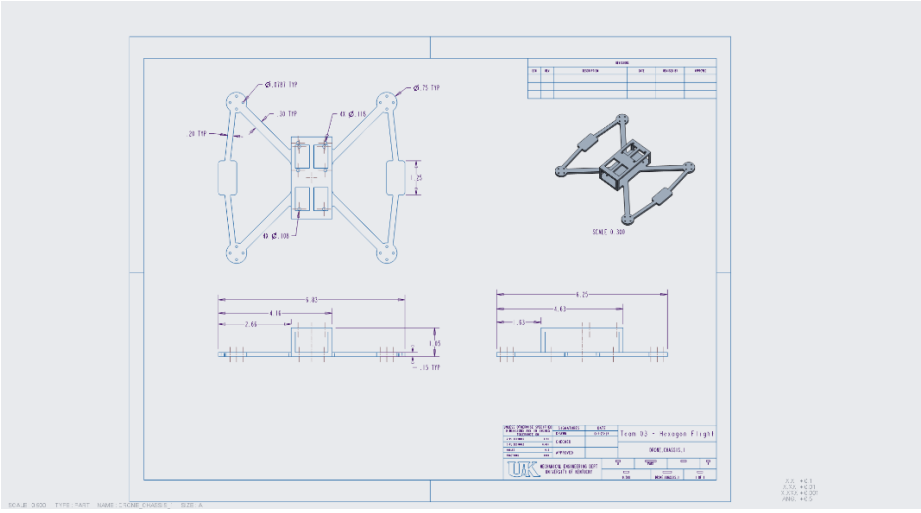


Figure 9. Drone Airframe Preliminary Drawing

The final design will use PETG-CF as the airframe material, as it has a higher stiffness (Young's Modulus) and greater vibrational damping [8][9]. The previous team also noted difficulties with PLA through testing and integration. Although PETG-CF is slightly heavier, there is an adequate weight buffer that is accounted for.

Due to the weight requirement, it has been analytically determined that there will be negligible central deflection and thus will not be physically tested. However, the previous team noted difficulties with inconsistencies between the center of thrust and center of mass. The center of mass will be identified through CAD software and then verified once printed. Integration issues are inevitable but will be solved by the end of the semester through rapid prototyping through 3D printing.

### *5.3.2 Power System*

For the end of this semester, the initial Power System prototype will consist of an integrated hardware assembly featuring the structural frame, four brushless DC motors, an all-in-one (AIO) flight controller/ electronic speed controller (ESC), the primary battery system, and any other payload electronics. While only the motors and battery system constitute this subsection's direct prototype deliverables, full integration with all other electronic components is essential to evaluating functional performance. In this configuration, the four motors are arranged with diagonal symmetry, with two spinning clockwise and two counterclockwise, to balance rotational torque. The core functionality of this prototype is to safely distribute power from the energy storage system (a 4S Lithium-Polymer battery) and to translate control signals from the flight controller into variable thrust to achieve controlled flight. Propulsion control and power management are central to this subproject, as the AIO flight controller actively modulates signals to govern motor RPM and behavior, which directly control flight dynamics. This contributes directly to Engineering Requirements 1, 3, 4, 6, and 13, which define the drone's ability to achieve controlled flight and spatial maneuvering. These capabilities are foundational to reaching our high-level mission goal of traversing target gates. Without the proper motors, the level of flight required to complete these goals will not be possible.

While advanced computational tasks outside of basic flight control, such as the vision system and path planning, are excluded from this baseline subsystem prototype, the battery must successfully power all auxiliary onboard systems. The battery subsystem enables ER 17 and 21, which dictate flight duration and battery swap time constraints; the battery must be able to sustain a target 6-minute flight on a single charge.

To validate this design, empirical testing will verify both motor and battery performance against the relevant ERs. First, continuity and voltage testing will confirm that all onboard electronic components are correctly wired and receiving proper voltage and power distribution from the battery system. Controlled flight tests will then evaluate motor throttle response, thrust generation, and overall flight stability. Finally, continuous, timed flights will be used to measure the true flight duration achievable on a single battery charge under typical load conditions. This prototype demonstrates the overall project feasibility by proving that our integrated propulsion architecture can support the drone's mass while maintaining the baseline stability needed for autonomous operation, which is a critical milestone given the stability issues noted by the previous team.

To progress towards our operational goals, all primary motor and battery components have been selected and ordered. Our drone prioritizes extended, controlled endurance over high-speed agility. In line with this design strategy, we selected motors with a velocity constant of 2550 KV, a deliberate reduction from the previous team's 4800 KV motors. While higher KV ratings favor rapid acceleration, lower KV motors deliver higher mechanical torque and superior thermal efficiency under continuous loads. This lower KV rating allows up to step up propeller diameter from 3.0" to 3.5", generating higher static thrust and aerodynamic efficiency [10]. Additionally, we scaled the motor size up from 1407 to 2004 (20mm stator

diameter, 4mm stator height). A wider stator provides increased torque and power handling, addressing last year's stability and control issues [11].

Complementing the propulsion system, our energy storage system utilizes a 4S 850 mAh 150C LiPo battery. The 4S configuration reduces current draw across the system to minimize resistive heat losses, while the 850 mAh capacity optimizes runtime without adding excessive weight. Furthermore, the high 150C discharge rating prevents voltage sag during throttle spikes [12].

These motor and battery components were deliberately chosen to satisfy our technical requirements while remaining strictly below the 250 g weight limit. To support this goal, a weight budget has been outlined as shown in Table 4, including both the planned and actual allocations. The forecasted budget allocated 239.5 g, providing a 10.5 g buffer below the regulatory limit. Based on the physical data from our purchased components, the actual system mass comes to 238.2 g, also preserving a reasonable margin. The Power System subproject is directly responsible for the first three rows in the table. While the 2004 motors were slightly heavier than budgeted, this overage was fully offset by weight savings in the battery and propellers.

Table 4. Drone Weight Budget

| Component | Weight Budgeted (g) | Actual Weight (g) |
|---|---|---|
| Motors | 64 | 67.2 |
| Propellers | 12 | 9.4 |
| LiPo battery | 100 | 99 |
| AIO Flight controller | 10 | 9.4 |
| Flight sensors | 4 | 3 |
| BEC module | 0.5 | 0.5 |
| SBC | 15 | 15 |
| Race LED | 1 | 0.7 |
| Camera | 8 | 9 |
| Frame and mounting hardware | 25 | 25 |
| **Total:** | **239.5** | **238.2** |

The system power budget as shown in Table 5 confirms that the selected battery will be capable of supporting the necessary electronic load. The pack's maximum current delivery of 127.5 A provides a high safety margin for the peak system draw of 32.6 A. Additionally, the battery has a total stored energy of 12.58 Wh, so based on the typical 129.56 W load, the drone is projected to achieve ~6-minute flight time in line with ER 17, with further endurance gains possible through active throttle management.

Table 5. System Power Budget

| Component | Operating Voltage (V) | Typical Current (A) | Peak Current (A) | Typical Power Consumption (W) | Peak Power Consumption(W) |
|---|---|---|---|---|---|
| Motors(x4) | 14.8 | 8 | 30 | 118.4 | 592 |
| AIO Flight Controller | 14.8 | 0.25 | 0.4 | 3.7 | 5.9 |
| Flight sensors | 5 | 0.12 | 0.19 | 0.6 | 0.95 |
| BEC module | 14.8 | 0.25 | 0.55 | 3.7 | 8.14 |
| SBC | 5 | 0.4 | 1 | 2 | 5 |
| Race LED | 5 | 0.2 | 0.4 | 1 | 2 |
| Camera | 5 | 0.03 | 0.06 | 0.16 | 0.3 |
| **Total Load:** | **-** | **9.25** | **32.6** | **129.56** | **614.29** |

### *5.3.3 Sensors*

The Sensors and Flight Stabilization subsystem will provide the flight controller with the information necessary to maintain stable flight and estimate the drone's position in a GPS-denied indoor environment. Unlike several other subsystems that are being redesigned, this subsystem will continue to use the same flight controller and sensor suite selected by the previous team. These components were retained because they successfully met the functional requirements of the original design and demonstrated reliable operation during testing. Documentation from the previous project showed that the Flywoo GOKU GN745 AIO flight controller, MTF-02P Optical Flow Sensor and VL53L0X Time-of-Flight sensor were successfully integrated and capable of providing valid sensor data to ArduPilot. Since the primary issues encountered by the previous team were related to flight stability, airframe balance, and controller tuning rather than sensor functionality, replacing these components would provide limited benefit while increasing development time and project risk. Reusing proven hardware allows the team to focus efforts on system integration and flight performance improvements instead of redeveloping already functional subsystems.




Figure 10. Flywoo GOKU GN745 AIO MicoAir MTF-02P optical flow sensor



Figure 11. MTF-02P Optical Flow Sensor and VL53L0X Time-of-Flight sensor

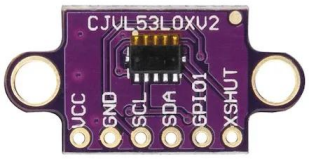



Figure 12. VL53L0X ToF Sensor

The selected flight controller is the Flywoo GOKU GN745 AIO, which contains an onboard Inertial Measurement Unit (IMU) consisting of gyroscopes and accelerometers used to measure angular velocity and acceleration along multiple axes. These measurements allow the autopilot to continuously control pitch, roll, and yaw by adjusting the motor outputs. To improve indoor navigation performance, the system utilizes the MicoAir MTF-02P optical flow sensor and the VL53L0X Time-of-Flight sensor. The optical flow sensor estimates horizontal movement by tracking patterns on the ground and measuring the displacement of image features between frames. This allows the drone to estimate velocity and position relative to the surface below. The VL53L0X ToF sensor uses laser ranging technology to measure the distance between the drone and the ground with millimeter-level resolution, providing accurate altitude information for takeoff, hovering, and landing operations.

The flight controller combines data from these sensors using the ArduPilot Extended Kalman Filter (EKF), which fuses measurements from multiple sources to provide a more accurate estimate of the drone's position and orientation than any single sensor could provide independently. This sensor fusion process is critical because optical flow measurements depend on accurate altitude data from the ToF sensor, while both measurements rely on the IMU to compensate for changes in attitude during flight. Together, these sensors provide the foundation for stable autonomous flight and future navigation capabilities.

To evaluate the first-semester prototype, the subsystem will undergo both bench testing and controlled flight testing. Sensor outputs will first be verified through Mission Planner to confirm accurate communication and calibration. The optical flow sensor will be tested by comparing measured motion against known distances traveled, while the ToF sensor will be evaluated using fixed-height measurements throughout the test enclosure. Following validation, flight tests will be conducted to evaluate hovering stability, altitude hold performance, and position estimation accuracy. Successful completion of these tests will demonstrate that the sensor suite can provide the reliable navigation and stabilization data required for future autonomous flight operations.

### *5.3.4 Autopilot*

The Autopilot and Single-Board Computer (SBC) subsystem serves as the central coordination hub of the autonomous drone. Operating hierarchically above the low-level flight controller unit (FCU), this subsystem manages mission state execution, stateful goal position estimation, kinematic trajectory generation, real-time obstacle avoidance, and telemetry distribution.

As established in the Level 1 Functional Decomposition (Section 9.1.2), the Autopilot subsystem interfaces directly across four major subsystem boundaries, working with the Vision, Flight, UI, and Power subsystems. From the Vision system, the autopilot receives goal positions and vision tag positions across a local Unix socket. The Vision system's camera is powered and communicated with via the CSI port on the SBC. The Flight subsystem and the Autopilot share a bidirectional MAVLink bridge over high-speed

UART, transmitting flight commands and vision localization data to the FCU, while receiving sensor data and system status back to the Autopilot. A high-speed wireless link with the UI allows for receiving sequence data and broadcasting telemetry and logs back to the users. The Power subsystem provides dedicated regulated 5V power lead from the power distribution system to power the SBC.

The goal for the first semester of development on the Autopilot is twofold: establishing a wireless proxy for ArduPilot for non-autonomous development and developing an Autopilot program that is drone-ready and can be integrated quickly once physical drone development and iteration is completed. The former goal requires that hardware is selected and acquired, and a preexisting proxy program is run for wireless testing. The latter goal centers around a simulation-tested program handling Autopilot logic, including data ingestion, goal estimation, and path planning and following. By the end of the semester, the drone should be able to fly wirelessly with the help of the ArduPilot proxy, and a simulated drone should run through the entire flight sequence with simulated inputs, ready for hardware testing. This separation ensures that development bottlenecks can be iterated independently, and the integration of the two systems hinges on a single, easily testable abstraction, enabling rapid integration early next semester to meet the E-Day demonstration deadline.

The Autopilot subsystem also encompasses the SBC device, which runs both the Autopilot and Vision programs. Several SBC boards were considered, and two stood out: the Raspberry Pi Zero 2W and the Radxa Zero 3W, which are compared in Table 6 below.

Table 6. Single-Board Computer (SBC) Selection

| Parameter | Raspberry Pi Zero 2W | Radxa Zero 3W (Selected) | Technical Impact on Autonomous Mission |
|---|---|---|---|
| **System-on-Chip (SoC)** | Broadcom BCM2710A1 | Rockchip RK3566 | 64-bit quad-core processing architecture |
| **CPU Microarchitecture** | 4x ARM Cortex-A53 @ 1.0 GHz | 4× ARM Cortex-A55 @ 1.6 GHz | Cortex-A55 provides $\sim 40%$ IPC improvement and 60% higher clock rate for vision processing |
| **RAM Capacity & Type** | 512 MB LPDDR2 (Shared) | 2 GB / 4 GB LPDDR4 | Critical: Resolves the primary memory exhaustion failure of the previous design team |
| **Memory Bandwidth** | 32-bit single-channel LPDDR2 | 32-bit LPDDR4 @ 2133 MT/s | Prevents frame drop when capturing 30 FPS global-shutter video into memory ring buffers |
| **Camera Interface** | 2-lane MIPI CSI-2 (Mini-connector) | 2-lane MIPI CSI-2 (Standard FPC) | Native high-speed camera interface |

| | | | |
|---|---|---|---|
| **Wireless Connectivity** | 2.4 GHz 802.11n WiFi + BLE 4.2 | 2.4 GHz / 5.0 GHz 802.11ac WiFi + BLE 5.0 | Dual-band support reduces RF congestion during live UI streaming at E-Day |
| **Physical Dimensions** | 65.0 x 30.0 x 5.0 mm | 65.0 x 30.0 x 5.0 mm | Identical mechanical footprint; fits directly in standard micro-carrier bays |
| **Bare Board Mass** | Roughly 10.0 g | Roughly 12.0 g | Sub-250 g budget allocation: <= 15.0 g with thermal heatsink and standoffs |
| **Power Consumption** | Roughly 1.5 W idle, roughly 3.2 W full load | Roughly 1.8 W idle, roughly 4.2 W full load | Powered via dedicated 5V/2A BEC output on power distribution board |
| **Operating System** | Raspberry Pi OS (Debian 32/64-bit) | Ubuntu Server 22.04 LTS (ARM64) | Full 64-bit modern Linux toolchain compatibility with C++17 and OpenCV 4.x |

Based on this evaluation, the Radxa Zero 3W was selected. In past senior design attempts, the 512 MB memory boundary of the Raspberry Pi Zero forced aggressive operating system pruning and led to kernel out-of-memory (OOM) panics during combined camera capture and navigation processing. The 2 GB to 4 GB LPDDR4 capacity of the Radxa Zero 3W provides substantial headroom for in-memory ring buffers, logging queues, and concurrent daemons.

The software stack is implemented in C++, designed around non-blocking asynchronous event loops and lightweight Inter-Process Communication (IPC), organized into five primary modules:

1. AutopilotNode: The master coordinating process executing the deterministic 20 Hz periodic loop (dt = 50ms), handling UI state changes, evaluating safety watchdogs, updating goal estimates, running the trajectory planner, and pushing setpoints to the FCU.
2. MavlinkDaemon: Manages interfacing the SBC with the low-level flight controller, maintaining bidirectional MAVLink 2.0 communication over hardware UART (/dev/ttyS0 at 921,600 baud), serializing flight commands, passing through vision localization data, and decoding incoming sensor data and system status.
3. GoalEstimator: A temporal state estimator processing raw frame-by-frame shape hypotheses from the vision pipeline, filtering spatial noise, tracking running covariance, and maintaining stable 3D coordinate representations of arena goals.
4. PathPlanner: A kinematic waypoint generator and obstacle avoidance solver implementing Artificial Potential Fields (APF) with repulsive vortex fields to navigate around non-target obstacles without local minima lockup.
5. UdpTelemetryLogger: A low-overhead diagnostic logger streaming live state telemetry over UDP to avoid blocking the primary control loop with disk I/O.

Because the SBC runs Linux, communication between independently developed processes is routed via Unix Sockets and local UDP endpoints, ensuring that subsystem modules remain language- and framework-agnostic. All IPC interfaces utilize compact, binary-packed C structures defined in shared/include/.

As established in the Level 1 Functional Decomposition, the Vision system transmits vision tag positions to the Autopilot via Unix socket, and the Autopilot communicates with the FCU via MAVLink over UART. The Autopilot forwards these world-space fiducial measurements directly to ArduPilot via MAVLink VISION_POSITION_ESTIMATE packets. This enables ArduPilot's internal EKF3 to zero its position drift dynamically, completely eliminating the need for separate physical wiring or dedicated bridge hardware between the camera and flight controller.

To validate data flow and pipeline timing prior to physical flight tuning, algorithmic proof-of-concept stubs were developed for goal tracking and path trajectory generation:

1. Stateful Goal Position Estimator: The vision system outputs frame-by-frame 3D coordinate hypotheses of detected gates relative to the camera lens. To reject image pixel noise and vibration artifacts, the GoalEstimator applies a first-order exponential temporal filter to maintain a persistent world-frame estimate:

$$\widehat{p_k} = \alpha p_{\text{meas},k} + (1-\alpha)\widehat{p_{k-1}} \tag{1}$$

where $\widehat{p_k} = [\widehat{x_k}, \widehat{y_k}, \widehat{z_k}]^T$ is the smoothed 3D position vector in the arena frame, $p_{\text{meas},k}$ is the instantaneous measurement transformed by vehicle attitude, and $\alpha = 0.5$ is the smoothing factor. The estimator tracks running spatial variance:

$$\sigma_k^2 = (1-\alpha)\sigma_{k-1}^2 + \alpha\left|p_{\text{meas},k} - \widehat{p_k}\right|^2 \tag{2}$$

Goals transition from SEARCHING (0) to CONVERGING (1) to LOCKED (2) when $\sigma^2 \leq 0.04$ m$^2$, and enter OCCLUDED (3) if measurements cease for more than 0.5s.

2. Path Planning & Artificial Potential Fields (APF): The path planner computes translational velocity vectors to guide the quadcopter through target gates while avoiding collisions with non-target gates. Trajectory generation establishes two primary waypoints per gate:

$$w_{\text{approach}} = \widehat{p_{\text{gate}}} - d_{\text{standoff}}\widehat{n_{\text{gate}}} \tag{3}$$

$$w_{\text{exit}} = \widehat{p_{\text{gate}}} + d_{\text{standoff}}\widehat{n_{\text{gate}}} \tag{4}$$

where $\widehat{p_{gate}}$ is the estimated gate center, $\widehat{n_{\text{gate}}}$ is the gate aperture unit normal vector, and $d_{\text{standoff}} = 0.50$ m is the approach distance. Attractive velocity towards the current waypoint $w$ is defined as , with gain . For non-target obstacles at distance $d_i \leq d_{\text{influence}}$ (), repulsive forces are computed:

$$v_{\text{rep},i} = k_{\text{rep}}\left(\frac{1}{d_i} - \frac{1}{d_{\text{influence}}}\right)\frac{r_i}{d_i^3} \tag{5}$$

To prevent collinear local minima traps, an orthogonal vortex swirl force is injected:

$$v_{\text{vortex},i} = k_{\text{vortex}}\left[-r_{y,i}, r_{x,i}, 0\right]^T \tag{6}$$

The total commanded velocity vector is clamped against physical kinematic limits:

$$v_{cmd} = clamp(v_{att} + \sum v_{rep,i} + \sum v_{vortex,i}, v_{max} \tag{7}$$

with $v_{\text{max}} = 2.0$ m/s horizontally and $v_{z,\text{max}} = 1.0$ m/s vertically, satisfying ER 3 and ER 6.

The safety monitoring architecture integrates multiple redundant watchdogs and interlocks to ensure reliable fault handling during flight. A vision loss watchdog continuously monitors incoming sensor datagrams, triggering an immediate emergency descent and motor cut if valid frames cease for more than 0.20 seconds. Similarly, the onboard flight controller processes a dedicated operator emergency stop command through wireless link protocols, executing an immediate thruster cutoff within 150 milliseconds of engagement. Finally, an automated low-voltage auto-land sequence monitors pack voltage telemetry via the flight controller status interface, initiating a controlled landing descent whenever operating voltage drops below critical cell thresholds under load.

### *5.3.5 Vision System*

The Vision System at the end of the semester will deliver functional shape and AprilTag vision target detection pipeline running onboard the drone's single-board computer (SBC). The software will interact directly with a carefully selected camera to provide the raw visual information needed. The basis for the shape detection is to use contour approximation to estimate vertices for geometric polygon identification. The vision target detection will use specially placed targets that can help localize the drone within its environment.

The biggest aspect of the selection criteria for the drone's camera was a global shutter image sensor. This type of sensor captures every pixel simultaneously rather than line-by-line like traditional rolling shutter sensors. A global shutter image sensor is preferred for our drone's camera because it eliminates the warping and skewing that comes from images taken during high-speed movement. This skewing can be seen below in Figure 13. Choosing a global shutter camera introduced the choice between a color or a monochrome sensor. If this were a project not constrained by a limited budget, a color global shutter camera would be selected and purchased. A color camera would allow the shape detection script to use the contour detection technique in tandem with a specific color hue range threshold. Practically, this means that the shaped targets could be made with a bright color that stands out in the environment and the shape detection would isolate this color for direct contour detection on the shapes. However, this is a project constrained by a limited budget and global shutter cameras in color are expensive. Ultimately, the OV9281 Global Shutter Camera Module was chosen for this project. This is a monochrome, 1 MP global shutter camera with a field of view of 79 degrees.

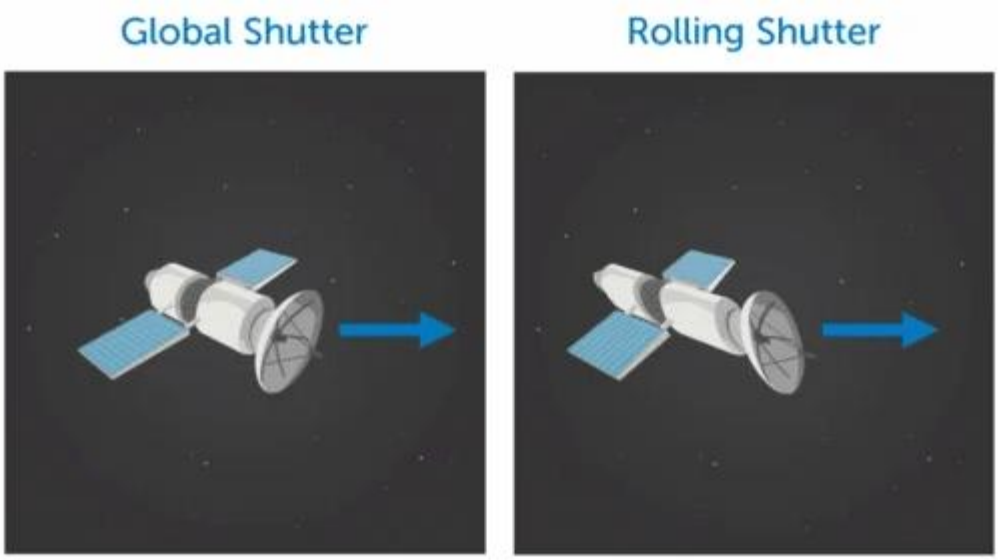



Figure 13. Object moving at high-speed comparison [13]

The shape detection script will make use of the OpenCV library [14]. Specifically, the script combines the use of Canny edge detection and findContours to draw a white outline of everything on a black background. Using the contour areas, the image can further be refined by throwing away small junk outlines. Next, approxPolyDP is used to count the number of vertices the extracted contour has. The output of this function is fed into an if else if list for shape identification. This method of contours to identify objects is traditionally unreliable with busy backgrounds, so it is best when paired with methods to reduce background noise like the hue filter with color cameras. To compensate for the lack of color from our camera, a new strategy was developed. This involves wrapping the targets in retroreflective tape and mounting a high-intensity white LED adjacent to the camera. By increasing the shutter speed, the normal room light fades to black and the only light source intense enough to register is the concentrated light bouncing off the tape from the white. This should create isolated, glowing shape contours that can be easily processed.

The vision system will need to send positional data to the autopilot for it to create the flight plans. Currently, only the distance to the gates is estimated using their known dimensions and the focal length of the camera. A complete 6 degrees of freedom pose estimation is being researched using the solvePnP algorithm. This algorithm will take in the known gate dimensions, the detected image points, an intrinsic camera matrix, and distortion coefficients to output the 3D translation and rotation vectors of the shaped gates. The drone's positional data will be calculated using vision targets on the net enclosure. The vision targets will each be associated to a different location on the net enclosure, so knowing the distance to these vision targets and scanning multiple will allow for accurate drone positional data.

To test the first-semester prototype, the vision system will undergo bench testing against physical target gates. The gate identification and pose estimation will be tested and verified for recognition accuracy, frame processing latency, and the precision of the output positional data. Additionally, the accuracy of the drone's positional data calculations will be verified using strategically placed vision targets.

### *5.3.6 User Interface*

By the end of the semester, the User Interface subsystem will have a functional prototype developed in Python using PySide6. An idea for the design can be seen in Figure 14. The interface will be designed for a demonstration during the University of Kentucky's Engineering Day. It will include a goal-selection panel for ordering the circle, triangle, and square in the user desired flight order, a camera-feed display, a telemetry panel with real-time information about the drone's status, mission status indicators, and Start mission and Emergency Stop buttons. The prototype will initially use simulated drone data to demonstrate the interface's functionality before full integration. The Level 2 functional decomposition (Figure 21, Section 9.1.8) for the UI will illustrate the subsystem's internal functions and the data exchanged between the operator, UI, and onboard single-board computer (SBC). It will include mission configuration, sequence transmission and acknowledgment, telemetry and status display, camera-feed display, and Emergency Stop handling. The associated IO table (Table 13) will identify the inputs and outputs for each interface and show how the UI connects with the Autopilot and Vision subsystems.

The UI will communicate with the SBC over Wi-Fi using network sockets. When the operator selects and confirms a goal sequence, the UI will package the ordered shapes into a structured message and transmit it to the Autopilot software running on the SBC. The Autopilot's state machine will receive the sequence and return an acknowledgment, allowing the UI to indicate whether the sequence was successfully received. The interface will also receive telemetry and mission-status updates from the onboard system and display the camera stream provided by the Vision subsystem.

The Emergency Stop button will send a stop command to the SBC, where the Autopilot's state machine will handle the emergency override and communicate with the flight controller through the MAVLink interface. The intended response is to disarm the motors and cut off their control outputs. Testing will begin with simulated data to verify sequence selection, message handling, acknowledgment responses, and error reporting. Following integration, tests will measure sequence acknowledgment latency, wireless range and packet loss, and the Emergency Stop response time. These tests will evaluate ER10, requiring motor-control output cutoff within 150 milliseconds; ER18, requiring sequence acknowledgment within 250 milliseconds with 100% acknowledgment; and ER19, requiring a wireless range of at least 10 meters with a packet drop rate below 5%. ER21, which specifies returning to flight-ready status within 120 seconds after a battery swap, will also be considered during system-readiness testing.

This prototype will demonstrate the feasibility of the UI by showing that an operator can configure a mission, transmit and confirm the selected sequence, monitor drone status, and initiate the required safety response through one interface. The Level 2 functional decomposition and IO table define the UI's functional boundaries and communication interfaces, helping the team integrate it with the SBC, Autopilot, Vision, and Flight Control subsystems. Testing first with simulated data and then with onboard hardware will help identify communication and control issues before full system integration and support the final E-Day demonstration.

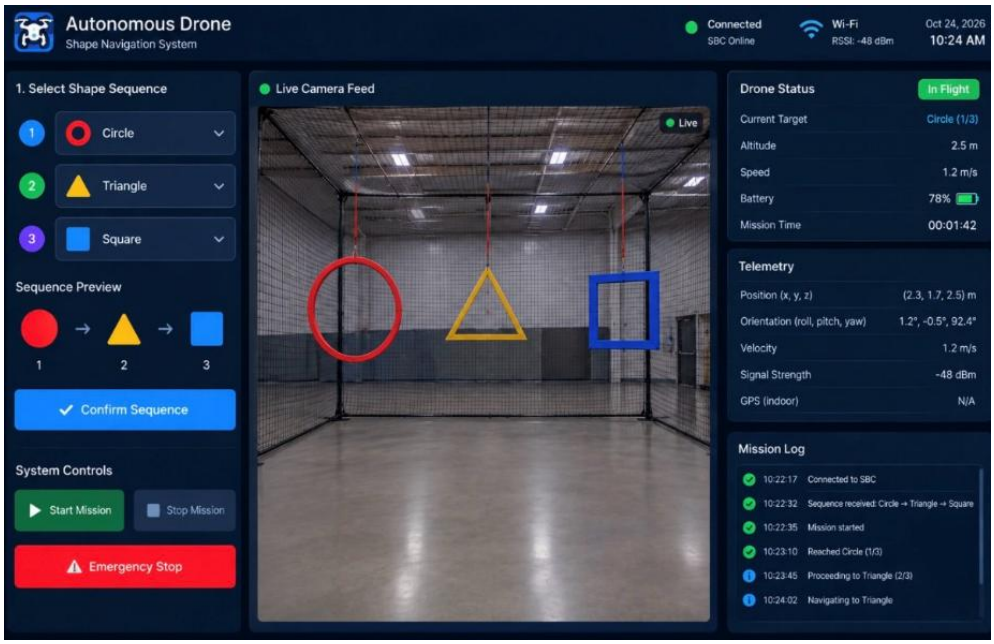



Figure 14. User Interface Design

### *5.3.7 Drone Course*

The drone course has been completed by the previous team, with a 6 x 6 x 6 ft area constructed from PVC pipe, which can be seen in Figure 15. The course will also be surrounded by netting for safety of the operators and spectators. There is also a bar along the top of course to connect the shapes too. The shapes will be modified to be 25% larger than the previous team's shape design and will also be equipped with reflective tape to make camera sensing easier. These shapes will be designed and 3D printed in PLA filament.



Figure 15. ShaperDrone Course Prototype

One of the limitations of the course is the limit on the position and orientation of the shapes within the enclosure. Solutions for this will be explored but are not a priority for this semester.

### *5.4 Initial Overall Prototype*

To integrate all individual subsection prototypes into a cohesive system, our initial prototype will feature a lightweight, 3D-printed PLA airframe designed for structural stiffness and vibrational damping. This will house a 4S 850 mAh 150C LiPo battery, a Flywoo GOKU GN745 AIO flight controller, and four 2004, 2550 KV BLDC motors arranged with diagonal symmetry to optimize thrust and thermal efficiency. Drone stability and precise indoor navigation will be achieved by combining real-time IMU data with a MicoAir MRF filter. The coordination hub and vision system will be driven by an onboard Radxa Zero 3W single board computer running a modular C++ software stack. This stack processes input from an OV9281 global shutter monochrome camera to execute 3D goal position estimation and collision-free flight path planning. The operation of the drone and system oversight will be managed through a PySide6-based User Interface which enables remote goal-sequence selection, real-time telemetry monitoring, and an emergency stop function capable of cutting motor controls within 150 ms.

The integrated prototype functions as a fully autonomous drone platform designed to navigate a 6x6x6 ft net enclosure by coordinating power distribution, sensor readings, computer vision, and real time flight control loops. Because the primary obstacle for the previous team was flight instability, our main objective for this semester's prototype is establishing repeatable, controlled flight while validation onboard hardware can reliably detect target shapes and relative distances. A full autonomous flight sequence across mission targets is intentionally reserved for the following phase, as establishing robust aerial stability is a mandatory prerequisite for full autonomy.

This will be validated using a multi-tiered approach combining rigorous bench testing to verify electrical wiring continuity and voltage regulation, calibrate Mission Planner sensors, test vision processing and distance estimation against physical targets, and evaluate UI logic with simulated telemetric data. This will be followed by controlled flight testing to measure hovering capabilities, altitude-retention, and continuous flight endurance relative to our 6-minute target. Successfully completing these milestones will demonstrate the feasibility of our project, confirming that we have resolved the physical control issues of the previous team and established a reliable hardware foundation for full autonomous deployment.

## 6 Project Management

### *6.1 Team Lead Information*

The HEXAGON Flight drone will have the following seven individual subsystems: Airframe, Power System, Sensors, Autopilot, Vision System, User Interface, and Drone Course. The breakdown of the design process into individual subsystems allows for systematic approaches to each aspect of the drone led by individual team members. Final integration will combine these separate components into a fully autonomous system. A summary of all team lead information along with the engineering requirements involved can be found below in Table 7.

Table 7. Team Lead Summary

| Team Lead Summary | | |
|---|---|---|
| **Subsystem** | **Team Lead** | **Involved ERs** |
| Airframe | Griffin Liem | ER 2, 5, 8, 11, 12, 20 ,21 |
| Power System | Ellie Carter | ER 1, 3, 4, 6, 13, 17, 21 |
| Sensors | Angel Landaverde | ER 1, 3, 15, 16, 22 |
| Autopilot | Rylan Moseley | ER 7, 9, 10, 18, 19, 20, 23 |
| Vision System | Bo Adams | ER 7, 14, 15, 16, 20 |
| User Interface | Kathryn Chumbley | ER 10, 18, 19, 21 |
| Drone Course | Griffin Liem | ER 24 |

The first subsystem is the Airframe, which is responsible for connecting all the other subsystems. As the airframe lead, Griffin will design it to have mounts for the flight controller/esc unit, raspberry pi, camera, motors, and sensors. This subproject is crucial because without it, the drone cannot be built and therefore cannot fly. For this semester, the final design of the airframe is expected to be completed and printed. The airframe will target engineering requirements 2, 5, 8, 11, 12, 20, 21. This subsystem will work closely with the power system, autopilot, sensors and camera to ensure proper mounting and integration.

The second subsystem is the Power System. This subsystem will be led by Eleanor Carter. For the autonomous drone to achieve any of its high-level mission goals, it must first establish reliable onboard power for critical electronic components and achieve stable, controlled flight. This subproject encompasses the full lifecycle of energy storage, thrust generation, and basic flight control hardware integration. The main responsibilities of this subsystem are selecting and integrating a high-efficiency battery capable of handling all electrical loads, distributing power at proper voltage levels, and sustaining sufficient battery life to support will ensure that the battery selected to power the drone will be capable of handling all electrical operations, distributing at the proper levels, and maintain sufficient battery life for the 2 minute flight duration. Also, Eleanor will select and integrate four brushless DC motors to integrate with the all-in-one flight controller/ESC unit and ensure the drone can execute a controlled lift off the ground and achieve stable, responsive flight dynamics. The Power System will specifically be focusing on engineering requirements 1, 3, 4, 6, 13, 17, and 21 covering key performance metrics such as power delivery, thrust, flight duration, and circuit safety. This subsystem will work closely with the Airframe system to establish motor mounting configurations and frame integration, as well as with the remaining electronic subsystems to ensure reliable power distribution across all onboard components.

The third subsystem is the sensors and flight stabilization led by Angel Landaverde, is responsible for providing the flight controller with the information necessary to maintain stable and controlled flight. Using onboard sensors such as the inertial measurement unit (IMU), the flight controller continuously monitors the drone's orientation and motion to regulate pitch, roll, and yaw while adjusting motor speeds to maintain balance and stability. Without accurate sensor measurements and proper calibration, the drone cannot effectively hover, maneuver, or fly safely. In addition to internal sensors, external sensors such as Time-of-Flight (ToF) sensors are used to measure distance from the ground and nearby obstacles, allowing the system to estimate altitude, detect hazards, and determine when to stop ascending.

This subsystem is a critical component of the overall project because autonomous flight depends on accurate position estimation and real-time environmental awareness, particularly in indoor environments where GPS is unavailable. The subsystem works closely with the autopilot, power, and propulsion subsystems by supplying sensor data and system status updates that support navigation and flight control decisions. This semester, the primary deliverables include evaluating sensor alternatives, integrating and calibrating selected sensors, and demonstrating stable hovering and smooth flight within the test environment. The subsystem directly supports Engineering Requirements 1, 3, 15, 16, and 22, which address flight safety, stability, position tracking, navigation performance, and successful completion of the project objectives.

The fourth subsystem is the Autopilot, which encompasses all high-level computational autonomy, flight mission supervision, path generation, target tracking estimation, and embedded inter-process communication. As the Autopilot Lead, Rylan is responsible for architecting the software hierarchy that translates high-level operator commands and computer vision detections into deterministic, dynamically feasible flight trajectories for the low-level flight controller. The key technical domains for this subsystem include SBC operating system bring-up and kernel tuning on the Radxa Zero 3W running embedded Linux, zero-copy binary communication protocols, and deterministic finite state machine management. Additional responsibilities involve stateful target goal estimation with exponential filtering, kinematic artificial potential field trajectory planning with vortex swirl mechanics, high-speed MAVLink UART bridge daemon execution, and compute memory budget management.

This subproject is crucial because it provides the core intelligence and coordination required for the drone to successfully navigate and execute its autonomous mission without external tracking infrastructure. For the first semester, deliverables include shared inter-process communication protocol headers, a cross-compilation CMake toolchain, a Software-in-the-Loop 3D simulator, a passing unit test suite, and benchtop UART MAVLink communication with the flight controller. Second-semester deliverables encompass physical airframe installation and cabling, closed-loop camera vision integration, in-cage state estimator and trajectory tuning, live autonomous demonstration at University of Kentucky E-Day (Saturday, February 27, 2027), post-E-Day robustness tuning, final evaluated capstone flight demonstrations (April 2027), and Senior Design Day hand-off (April 30, 2027). The assigned Engineering Requirements comprise primary technical ownership of ER 7, ER 9, ER 10, ER 18, ER 19, ER 20, and ER 23. This subsystem works closely with the Vision system to ingest target coordinates, the Flight subsystem to command actuator setpoints, the User Interface to receive sequence data and transmit telemetry, and the Power system to maintain stable 5V board regulation.

The fifth subsystem, the Vision System, will be led by Bo Adams. Given the autonomous nature of the drone, a means to take in and process information about its surrounding environment is required. This critical job will be completed with the Vision System. Its primary function will be to provide accurate positional data for the drone and the target-shaped gates to the autopilot for flight planning. This positional data will be acquired using written software for Shape and Vision Tag Detection. For the semester, the software is expected to be written and fully implemented on the single board computer to work with the drone’s camera. The Vision System will target engineering requirements 7, 14, 15, 16, and 20. This subsystem will work closely with the autopilot to ensure successful flight navigation.

The sixth subsystem is the User Interface (UI), which will be led by Kathryn Chumbley. The purpose of this subsystem is to provide an accessible interface for operating and monitoring the autonomous drone during the University of Kentucky's Engineering Day demonstration. The UI will be developed in Python using PySide6 and will allow users to select the desired flight sequence of the circle, triangle, and square before starting a mission. It will also display the camera feed, telemetry, and mission status, along with Start Mission and Emergency Stop buttons. The interface will communicate wirelessly with the single-board computer (SBC) to submit the selected sequence and confirm its receipt before the mission begins. The Emergency Stop will allow the operator to initiate the drone's safety response if needed. This semester, the UI prototype will be developed and tested using simulated drone data before integration with the physical system. The UI subsystem will specifically focus on engineering requirements 10, 18, 19, and 21 and work closely with the SBC/Autopilot and flight-control subsystems to support reliable mission operation and provide clear information to the operator.

The seventh and final subsystem is the Drone Course, which will be led by Griffin Liem. The purpose of this subsystem is to create a safe testing area for design verification and E-Day demonstration. The overall infrastructure of the course has been developed by the previous team, however, the shapes and connections will be modified. The shapes will be lined with reflective tape for improved vision processing, and the connections will be designed to be more robust to allow for more unique shape configurations. The shapes will also be slightly larger to accommodate for the increased drone size. The drone course will target engineering requirement 24 and will work closely with all subsystems once fully integrated.

### *6.2 Work Breakdown Structure (WBS)*

A Work Breakdown Structure displays and defines the product to be developed. It is a tool in project management used to break down complex tasks into smaller, more manageable tasks. Both semesters' deliverables, individual sub-project tasks, and estimated timelines are detailed below in our team's WBS in Table 8.

Table 8. Work Breakdown Structure

| ID | Activity | Description | Deliverables / Checkpoints | Duration (days) | People | Resources | Predecessors |
|---|---|---|---|---|---|---|---|
| **1** | **Chassis & Enclosure** | | | | | | |
| 1.1 | Design Chassis | Complete the design and verify center of mass calculations | • CAD model<br>• Center of Mass at origin of part | 14 | Griffin | PC, Creo Parametric | |
| 1.1.1 | 3D Print Chassis | 3D print all components of the chassis | • 3D printed chassis<br>• 3D printed landing gear | 7 | Griffin | 3D printer, PLA filament | 1.1 |
| **1.2** | **Construct Chassis** | Assemble complete chassis (without electronics) | | | | | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1.2.1 | Combine chassis and landing gear | Use nuts and bolts to connect the landing gear to the ends of the chassis by the motor mounts | • Fully assembled Chassis<br>• Remains level on flat surface | 3 | Griffin | M_ Nuts and Bolts, 3D printed parts | 1.1.1 |
| 1.2.2 | Integrate electronic components | Verify that the mounts for all of the electronics fit and secure their respective components | • Complete Drone Assembly | 3 | Griffin | Nuts and Bolts, Electronic Components, Chassis Assembly | 1.2.1 |
| 1.3 | Design shapes | design required shapes that the drone will fly though | • CAD models | 7 | Griffin | PC, Creo Parametric | |
| 1.3.1 | 3D print shapes | 3D print the design shapes | • 3D printed circle, square, and triangle | 3 | Griffin | 3D printer, PLA filament | 1.3 |
| 1.4 | Construct enclosure with shapes | Connect PVC pipe and set up the shapes | • Fully connected enclosure with netting<br>• Shapes connected to enclosure | 7 | Griffin | | |
| **2** | **Power and Propulsion** | | | | | | |
| **2.1** | **Obtain 4 motors** | **Procurement phase for brushless motors** | | | | | |
| 2.1.1 | Research and align motor specs with project goals | Select optimal motor KV and thrust to match frame and payload | • Motor specification sheet<br>• Compatibility report | 7 | Ellie | PC, Manufacturer data sheets | 1.1 |
| 2.1.2 | Purchase 4 motors | Order and receive 4 matched motors from supplier | • 4x Brushless DC motors received | 10 | Ellie | Online retailer, project budget | 2.1.1 |
| **2.2** | **Obtain batteries** | **Procurement phase for energy storage system** | | | | | |
| 2.2.1 | Research and align motor battery specs with project goals | Determine optimal voltage and discharge rating for required flight time and electronic loads | • Battery capacity & cell rating specification | 3 | Ellie | PC, Flight time calculation tools | 2.1.1 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.2.2 | Purchase 2 batteries | Order and receive primary and secondary LiPo batteries along with a compatible charger | • 2x 4S LiPo batteries received | 10 | Ellie | Online retailer, project budget | 2.2.1 |
| **2.3** | **Integrate components** | **Physical installation and electrical linking phase** | | | | | |
| 2.3.1 | Mount motors on chassis | Secure 4 motors to the 3D printed chassis motor mounts using proper fasteners | • Motors physically mounted and torqued down | 2 | Ellie and Griffin | Motors, M2/M3 screws, Hex keys | 2.1.2 |
| 2.3.2 | Connect battery to chassis | Secure battery mounting position and route power leads safely to the flight controller | • Battery securely strapped to chassis<br>• Power leads routed without tension | 2 | Ellie and Griffin | Battery strap, zip ties, flight controller | 2.2.2 |
| 2.3.3 | Link motors to flight controller | Connect flight controller and motor wires and verify continuous electrical connection | • Electrical continuity verified via multimeter<br>• flight controller signal wires connected to flight controller | 3 | Ellie and Angel | Soldering iron, wire strippers, multimeter | 2.3.2 |
| 2.3.4 | Tune motors for stable flight | Configure motor protocols and verify correct motor spin directions | • Correct motor rotation verified in configuration software<br>• Test bench safety check complete | 5 | Ellie and Angel | Configuration software, PC, USB cable | 2.3.3 |
| **3** | **Flight Control** | | | | | | |
| **3.1** | Configure Flywoo GOKU GN745 with ArduPilot and verify operation. | Setup Flywoo GOKU GN745 with ArduPilot firmware and verify operation. | • FCU connection established<br>• sensor ports verified | 2 | Angel | PC, Mission Planner, GN745 | |
| **3.2** | Integrate MTF-02P Optical Flow and VL53L0X ToF sensors. | Connect and configure VL53L0X Tof and MTF-02P Optical Flow sesnors. | • Sensors detected<br>• Data available in Mission Planner | 2 | Angel | GN745, Sensors, Mission Planner | 3.1 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **3.3** | Sensor testing and validation. | Verify sensor accuracy and functionality. | • Distance measurements recorded<br>• Optical flow data verified | 5 | Angel | Test Area, Mission Planner | 3.2 |
| **3.4** | Configure EKF parameters. | Configure EKF sources and sesnors fusion settings for indoor navigation. | • EKF parameters completed<br>• Stable position estimate achieved | 4 | Angel | Mission Planner, ArduPilot Documentation | 3.3 |
| **3.5** | Flight testing and tuning. | Conduct hover tests and tune controller parameters. | • Stable flight achievied<br>• Flight logs collected, oscillations minimized | 7 | Angel | Drone, Battery, Safety Area | 3.4 |
| 3.5.1 | Hover stability and testing. | Test altitude hold and position hold indoors. | • Flight data logs<br>• Hover performance metrics | 3 | Angel | Drone Test Area | 3.4 |
| 3.5.2 | Sensor Assisted Navigation Testing | Test optical flow and ToF performance during movement. | • Postion tracking verified<br>• Altitude tracking verified | 3 | Angel | Drone Test Area | 3.5.1 |
| 3.5.3 | EKF Optimization | Refine EKF settings based on flight logs. | • Reduced estimation error<br>• Updated parameters | 2 | Angel | Flight Logs, Mission Planner | 3.5.2 |
| **3.6** | System integration verification | Validate all navigation systems together. | • Complete subsystem demonstration | 7 | Angel | Fully Assembled Drone | 3.5.3 |
| **3.7** | Autonomous flight validation | Verify drone can maintain stable indoor flight suitable for obstacle navigation. | • Successful autnomous flight<br>• Final test documentation | 7 | Angel | Assembeled Drone, Test Course | 3.6 |
| **4** | **Autopilot** | | | | | | |
| **4.1** | Specify and purchase SBC | Choose a hardware board for the drone autonomous systems | • purchased SBC board | 2 + shipping time | Rylan | | |
| **4.2** | Configure SBC OS and peripherals | Setup the single-board computer for drone usage | • Appropriate Linux distro for board installed, configured for UART, camera, WiFi AP | 2 | Rylan | SBC | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **4.3** | Setup ARM cross-building for SBC software project | Configure the software build system for SBC compatibility | • CMAKE configuration written and tested<br>• Test program run on SBC<br>• Automatic CI builds configured | 2 | Rylan | SBC, GitHub | |
| 4.4 | Design and build autopilot test environment | Design an environment with simulated inputs to the autopilot program in order to verify output | • 3D environment and visualization for simulated testing<br>• Inputs to Autopilot program derived from stubs/simulations of other systems (Vision, UI, FCU)<br>• Outputs of Autopilot program visualized | 2 | Rylan | Software Simulation | |
| 4.5 | Write and test MAVLink daemon | Build a daemon that can interface with the FCU via MAVLink over UART | • Relevant MAVLink messages can be sent to FCU<br>• All needed MAVLink messages are supported by daemon<br>• Heartbeat and lifecycle maintained automatically by daemon | 3 | Rylan | SBC, FCU | |
| 4.6 | Build flight testing program | Build an intermediate program that utilizes the MAVLink daemon to let the flight team wirelessly test the drone | • Program for wireless drone testing<br>• Ardupilot Proxy + Custom tests | 4 | Rylan | Drone | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.7 | Design goal estimator | Mathmatically formulate a temporal filtering system for goal measurements in the simulation environment | • Filter selection for goal estimator<br>• Theoretical gains for goal estimator | 2 | Rylan | Software Simulation | |
| 4.8 | Design path planner | Mathmatically formulate a flight path generator based on drone inputs | • Point-to-point path planner system and visualizer | 2 | Rylan | Software Simulation | |
| 4.9 | Implement and test goal estimator | Implement the goal estimator in software and run it in the test environment | • Goal estimator integrated in testing environment | 3 | Rylan | Software Simulation | |
| 4.10 | Implement and test simple path planner | Implement a point-to-point path planner in software and run it in the test environment | • Path planner integrated in testing environment | 3 | Rylan | Software Simulation | |
| 4.11 | Implement Vision integration | Plumb the vision data ingress for the autopilot | • Vision data input to Autopilot program | 2 | Rylan | Software Simulation | 7.2 |
| 4.12 | Implement UI integration | Plumb the UI data ingress and outputs | • UI data input and output with Autpilot program | 2 | Rylan | Software Simulation | 7.1 |
| 4.13 | Implement and test optimized path planner | Implement a higher-order path planner for a more complex flight | • Spline or time optimized path planner and visualizer | 4 | Rylan | Software Simulation | 4.8 |
| 4.14 | Install to drone and integrate with other systems | Physically install the SBC on the drone and test with all systems connected | • SBC installed to drone<br>• Autonomous system functional | 10 | Rylan | Drone | 4.1-13, 7.3 |
| **5** | **Vision System** | | | | | | |
| **5.1** | **Develop Shape Detection** | | | | | | |
| 5.1.1 | Implement Contour and Shape Classifier | Develop contour approximation and geometric classification for target shapes | • Classification script<br>• Frame screenshots of identified shapes | 7 | Bo | PC, OpenCV | |
| 5.1.2 | Extract Centroid and Contour Metrics | Calculate centroid and contour area for each detected shape | • Centroid extraction script | 4 | Bo | PC, OpenCV | 5.1.1 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.1.3 | Extract Distance to and Centroid Position Relative to Camera | Use focal length to estimate distance to target and target centroid error to camera center | • Distance and centroid error script<br>• Test results from different target positions | 4 | Bo | PC, OpenCV, Shaped Targets | 5.1.1, 5.1.2 |
| 5.1.4 | Test Shape Detection and Troubleshoot | Test detection accuracy, false-positive rate, execution time, and extracted data | • Test results and verified functions | 4 | Bo | PC, OpenCV, Shaped Targets | 5.1.1, 5.1.2, 5.1.3 |
| **5.2** | **Vision Tags Detection** | | | | | | |
| 5.2.1 | Design & Print Vision Tags | Determine tag family, generate high-contrast markers | • Vision tags to be mounted on enclosure | 2 | Bo | PC, Chosen Vision Tags | |
| 5.2.2 | Vision Tag Detection & Identification | Isolate tags from incoming video frames and identify for drone localization | • Tag Detection code<br>• Test results for recognition with varying angles | 4 | Bo | PC, Vision Tags | 5.2.1 |
| 5.2.3 | Extract Distance and Position Relative to Camera | Use focal length to estimate distance to tag and position relative to tag | • Position Estimation Script<br>• Test results from different target positions | 4 | Bo | PC, Vision Tags | 5.2.1, 5.2.2 |
| 5.2.4 | Test Vision Tag Detection and Troubleshoot | Test detection accuracy, false-positive rate, execution time, and extracted data | • Test results and verified functions | 4 | Bo | PC, Vision Tags | 5.2.1, 5.2.2, 5.2.3 |
| **5.3** | **Drone Camera** | | | | | | |
| 5.3.1 | Research Drone Camera and Purchase | Research drone camera that will be compatible with current drone components and fill out order form | • Acquire drone camera | 2 | Bo | PC | |
| 5.3.2 | Connect camera to SBC | Figure out all the camera drivers to install and get a live feed up and running | • Working Drivers<br>• Live Feed | 7 | Bo | PC, Camera, SBC | 4.1, 5.3.1 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.3.3 | Implement software to SBC and Camera | Write all software to the SBC to fully implement drone vision system | • Working Shape Detection on drone SBC using drone Camera<br>• Working Vision Tag Detection on drone SBC using drone camera | 7 | Bo, Rylan | PC, Camera, SBC | 4.1, 5.1, 5.2, 5.3.1, 5.3.2 |
| **6** | **User Interface** | | | | | | |
| 6.1 | Design UI layout | Plan the arrangement and appearance of the interface. UI mockup completed | • UI layout/mockup | 3 | Kathryn | PySide6, design software | |
| 6.2 | Implement Shape Selection | Develop the interface for selecting shapes and arranging their flight order | • Shape selection and ordering functionality | 2 | Kathryn | PySide6, Python | |
| 6.3 | Implement Mission Controls | Develop controls for starting and stopping a mission. | • Start Mission, Emergency Stop, and sequence validation | 3 | Kathryn | PySide6, Python | |
| 6.4 | Implement Mission Status and Logging | Display mission progress and record important system events. | • Mission status display and event log | 3 | Kathryn | PySide6, Python | |
| 6.5 | Implement camera feed | Camera display panel with video feed integration | • Camera panel created display | 2 | Kathryn | Python, camera hardware | |
| 6.6 | Implement Telemetry Display | Develop a panel for displaying drone status information | • Battery, connection, and flight status display | 3 | Kathryn | Python, simulated telemetry data | |
| 6.7 | Integrate SBC Communication | Connect the interface to the SBC to transmit mission sequences and receive acknowledgments and status updates. | • Establish communication between the interface and the onboard computer. Mission sequence transmitted; acknowledgment and status updates received. | 3 | Kathryn | Python, SBC | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.8 | Test and Validate UI | Evaluate interface functionality and prepare it for system integration. | • Test results and verified UI functions | 7 | Kathryn | Completed UI, camera, simulated telemetry data, SBC | |
| **7** | **System Integration & Flight Testing** | | | | | | |
| 7.1 | Define Autopilot/UI interactions | Specify data structures | • Shareable header files for IPC | 1 | Rylan, Bo, Kathryn | GitHub | |
| 7.2 | Define Autopilot/Vision interactions | Specify data structures | • Shareable header files for IPC | 1 | Rylan, Bo, Kathryn | GitHub | |
| 7.3 | First test flight with past drone | To better understand the systems, many of which we will be reusing, and to understand and address previous issues, we will conduct several test flights with the previous team's drone | • Lessons learned from flight tests | 5 | All | Old Drone, net, room reservation | |
| 7.4 | Assemble Drone | Compile and construct all components of the drone | | 5 | All | Drone components | |
| | Final drone test flight | Test and verification flights (ER Verification) | • ER verification proof, test flight data | 15 | All | Drone | 7.4 |
| **8** | **Fall Semester Deliverables** | | | | | | |
| 8.1 | Biweekly Individual Project Reports | Reports to be completed individually and biweekly for the course of the semester | • Completed bieweekly individual project reports | | All | | |
| 8.2 | Sub-Project Progress Meetings | Meetings with the course professors and advisors, total of two | • Meetings completed | | All | | |

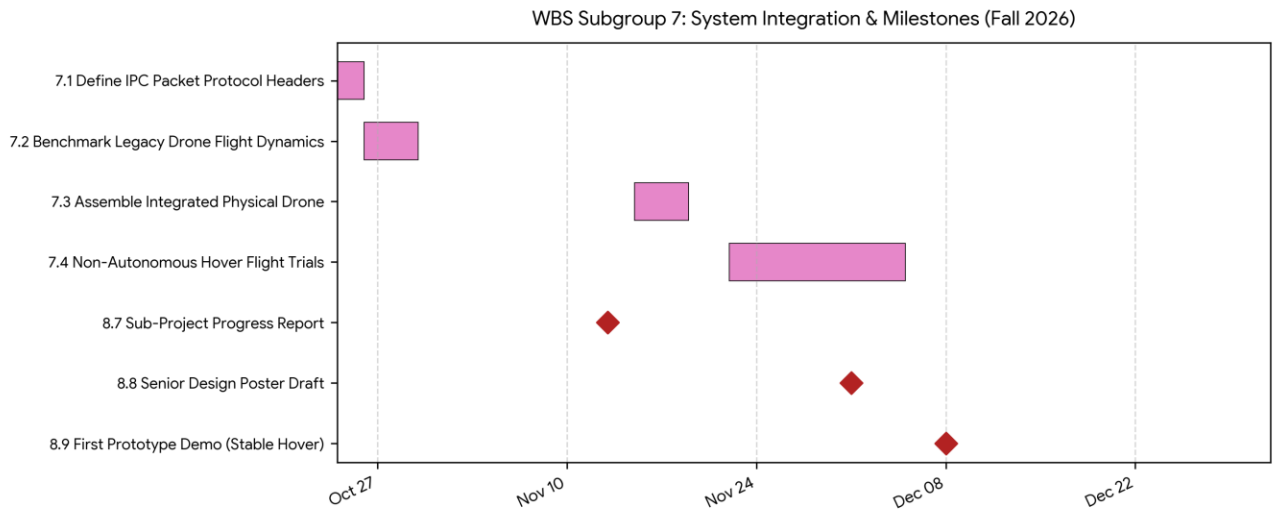
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 8.3 | Certificates of Participation | Assignment to report on team participation and evaluations | • Completed evaluations | | All | | |
| 8.4 | Requirements Presentation | Formal presentation of the user story, MRs, and Ers | • Requirement presentation given | | All | | |
| 8.5 | Functional Decomposition and Behavioral Models | Project management tools that break down complex tasks into smaller, more maneagable tasks | • Completed functional decompositions and behavioral models | | All | | |
| 8.6 | PDR Final | A description of an initial design/solution that satisfies the Ers | • Completed PDR final | | All | | |
| 8.7 | Sub-Project Progress Report | An individual report on assigned sub-projects | • Completed sub-preject progress report | | All | | |
| 8.8 | Senior Design Day Poster | A poster to be displayed at senior design day | • Completed poster | | All | | |
| 8.9 | First Prototype | The first prototype due date | • Assembled Prototype | | All | | |
| **9** | **Spring Semester Deliverables** | | | | | | |
| 9.1 | Biweekly Individual Project Reports | Reports to be completed individually and biweekly for the course of the semester | • Completed biweekly individual project reports | | All | | |
| 9.2 | Sub-Project Progress Meetings | Meetings with the course professors and advisors, total of two | • Meetings completed | | All | | |
| 9.3 | Certificates of Participation | Assignment to report on team participation and evaluations | • Completed evaluations | | All | | |
| 9.4 | CDR Final | Finalized and detailed technical design overview | • Completed CDR Final | | All | | |
| 9.5 | Final Presentations | Final design presentation | • Presentation completed | | All | | |

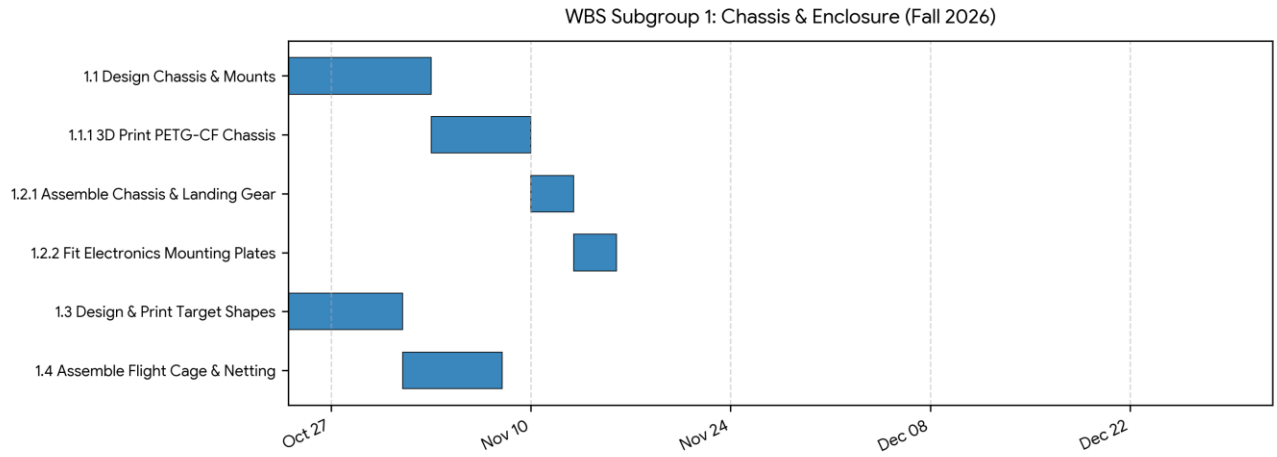
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 9.6 | Final Report, Poster | Complete project lifecycle and presentation poster | • Completed Final Report and poster | | All | | |
| 9.7 | E-Day Autonomous Flight Demo | Execute the full autonomous sequence for the public | • Successful demonstration | | All | | |

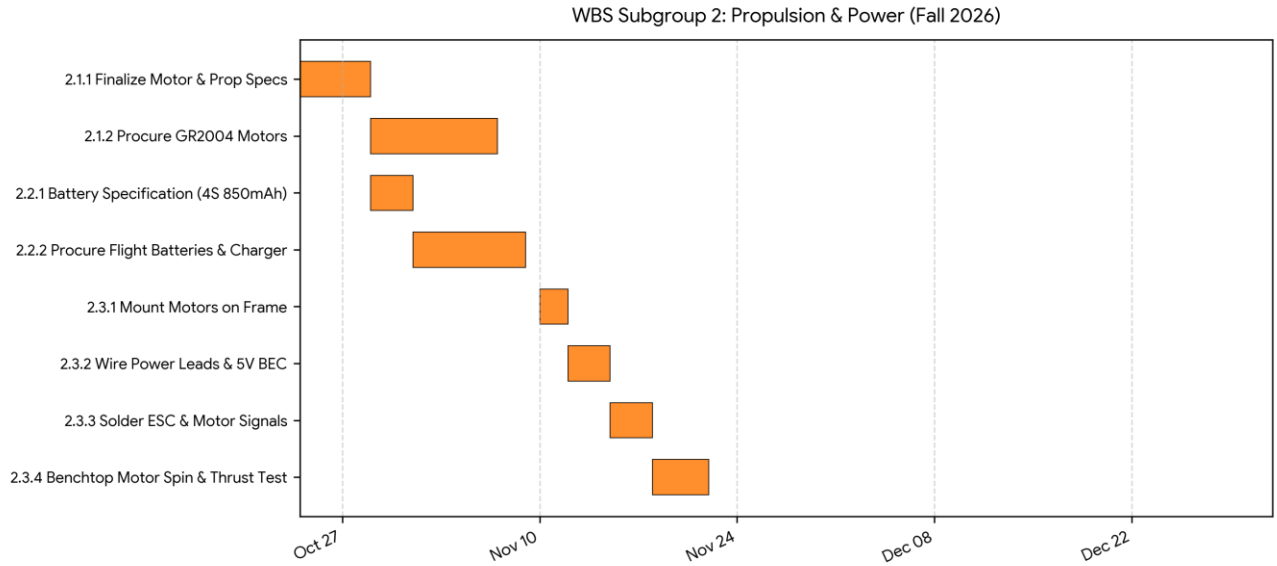
### *6.3 Gantt Chart*

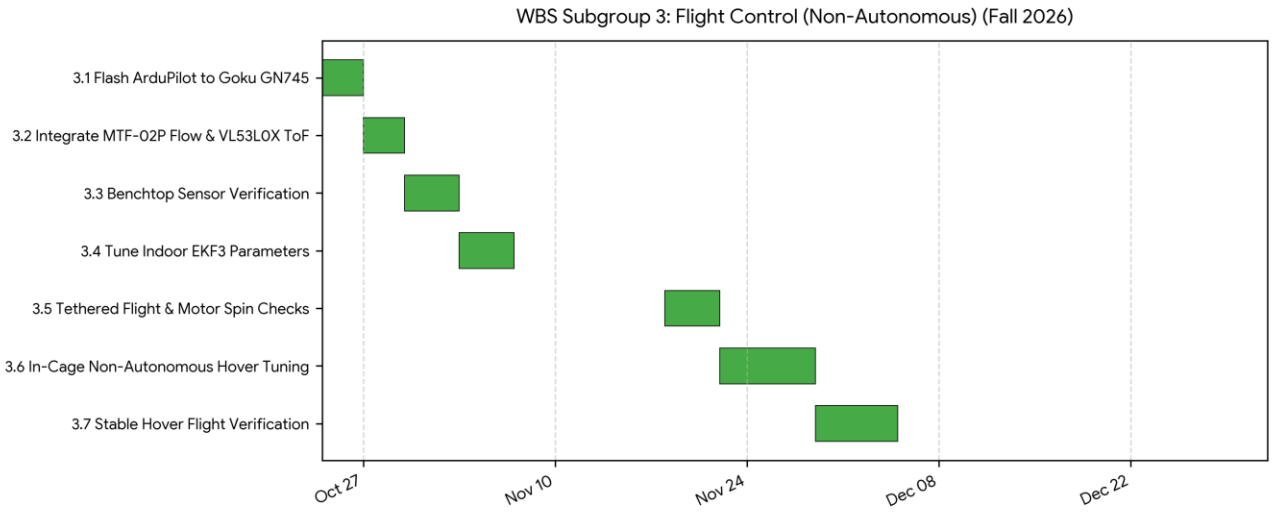
To manage schedule execution following the Preliminary Design Review, the project timeline is partitioned into seven dedicated subsystem Gantt charts derived directly from the Work Breakdown Structure (Table 8). The development roadmap reflects the primary objective for the first semester: delivering a fully integrated, non-autonomous flying quadcopter capable of stable indoor hover alongside independent benchtop and simulation prototypes for the computing subsystems.

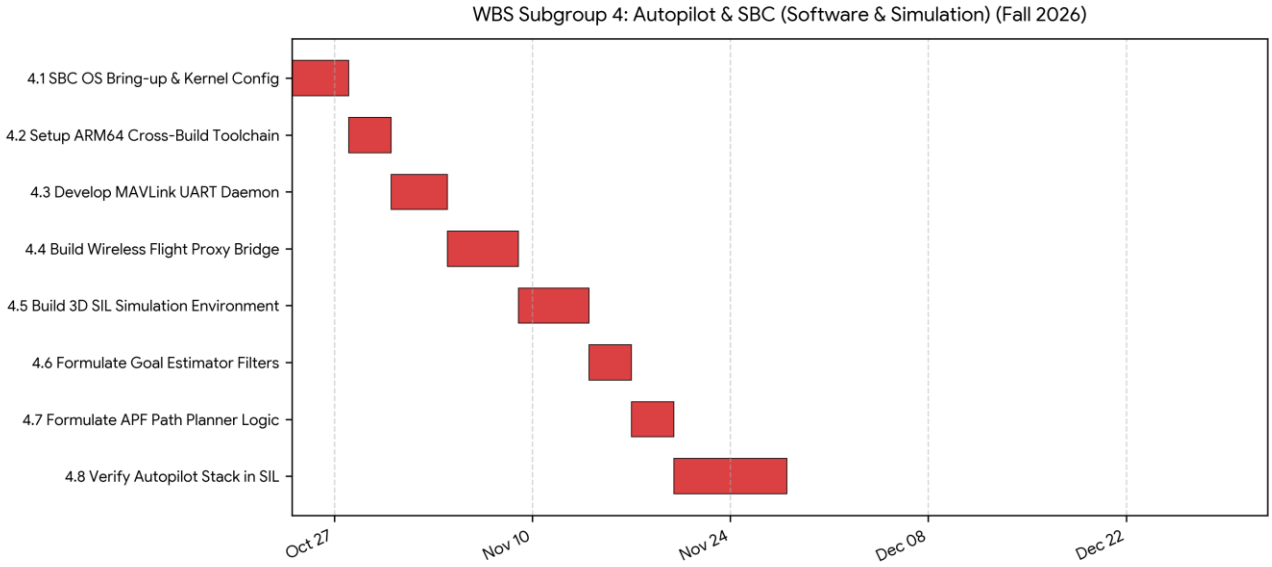
As illustrated in Figure 16, physical hardware efforts focus on 3D printing the PETG-CF airframe, assembling the propulsion and power distribution harness, bringing up ArduPilot on the Flywoo Goku GN745 flight controller, and conducting optical-flow-assisted hover tuning in the flight enclosure. Concurrently, the computational subsystems develop in isolation: the Autopilot software stack is verified within a 3D Software-in-the-Loop (SIL) simulation while providing a wireless proxy bridge for manual flight tuning; the Vision pipeline executes benchtop shape classification and AprilTag pose estimation; and the PySide6 User Interface is validated using mock telemetry streams. These parallel workstreams converge during late November for unified airframe assembly and tethered hover verification, satisfying the exit criteria for the First Prototype Demonstration on December 8, 2026.


WBS Subgroup 7: System Integration & Milestones (Fall 2026)
7.1 Define IPC Packet Protocol Headers
7.2 Benchmark Legacy Drone Flight Dynamics
7.3 Assemble Integrated Physical Drone
7.4 Non-Autonomous Hover Flight Trials
8.7 Sub-Project Progress Report
8.8 Senior Design Poster Draft
8.9 First Prototype Demo (Stable Hover)
Oct 27
Nov 10
Nov 24
Dec 08
Dec 22


WBS Subgroup 1: Chassis & Enclosure (Fall 2026)
1.1 Design Chassis & Mounts
1.1.1 3D Print PETG-CF Chassis
1.2.1 Assemble Chassis & Landing Gear
1.2.2 Fit Electronics Mounting Plates
1.3 Design & Print Target Shapes
1.4 Assemble Flight Cage & Netting
Oct 27
Nov 10
Nov 24
Dec 08
Dec 22


WBS Subgroup 2: Propulsion & Power (Fall 2026)
2.1.1 Finalize Motor & Prop Specs
2.1.2 Procure GR2004 Motors
2.2.1 Battery Specification (4S 850mAh)
2.2.2 Procure Flight Batteries & Charger
2.3.1 Mount Motors on Frame
2.3.2 Wire Power Leads & 5V BEC
2.3.3 Solder ESC & Motor Signals
2.3.4 Benchtop Motor Spin & Thrust Test
Oct 27
Nov 10
Nov 24
Dec 08
Dec 22


WBS Subgroup 3: Flight Control (Non-Autonomous) (Fall 2026)
3.1 Flash ArduPilot to Goku GN745
3.2 Integrate MTF-02P Flow & VL53L0X ToF
3.3 Benchtop Sensor Verification
3.4 Tune Indoor EKF3 Parameters
3.5 Tethered Flight & Motor Spin Checks
3.6 In-Cage Non-Autonomous Hover Tuning
3.7 Stable Hover Flight Verification
Oct 27
Nov 10
Nov 24
Dec 08
Dec 22


WBS Subgroup 4: Autopilot & SBC (Software & Simulation) (Fall 2026)
4.1 SBC OS Bring-up & Kernel Config
4.2 Setup ARM64 Cross-Build Toolchain
4.3 Develop MAVLink UART Daemon
4.4 Build Wireless Flight Proxy Bridge
4.5 Build 3D SIL Simulation Environment
4.6 Formulate Goal Estimator Filters
4.7 Formulate APF Path Planner Logic
4.8 Verify Autopilot Stack in SIL
Oct 27
Nov 10
Nov 24
Dec 08
Dec 22

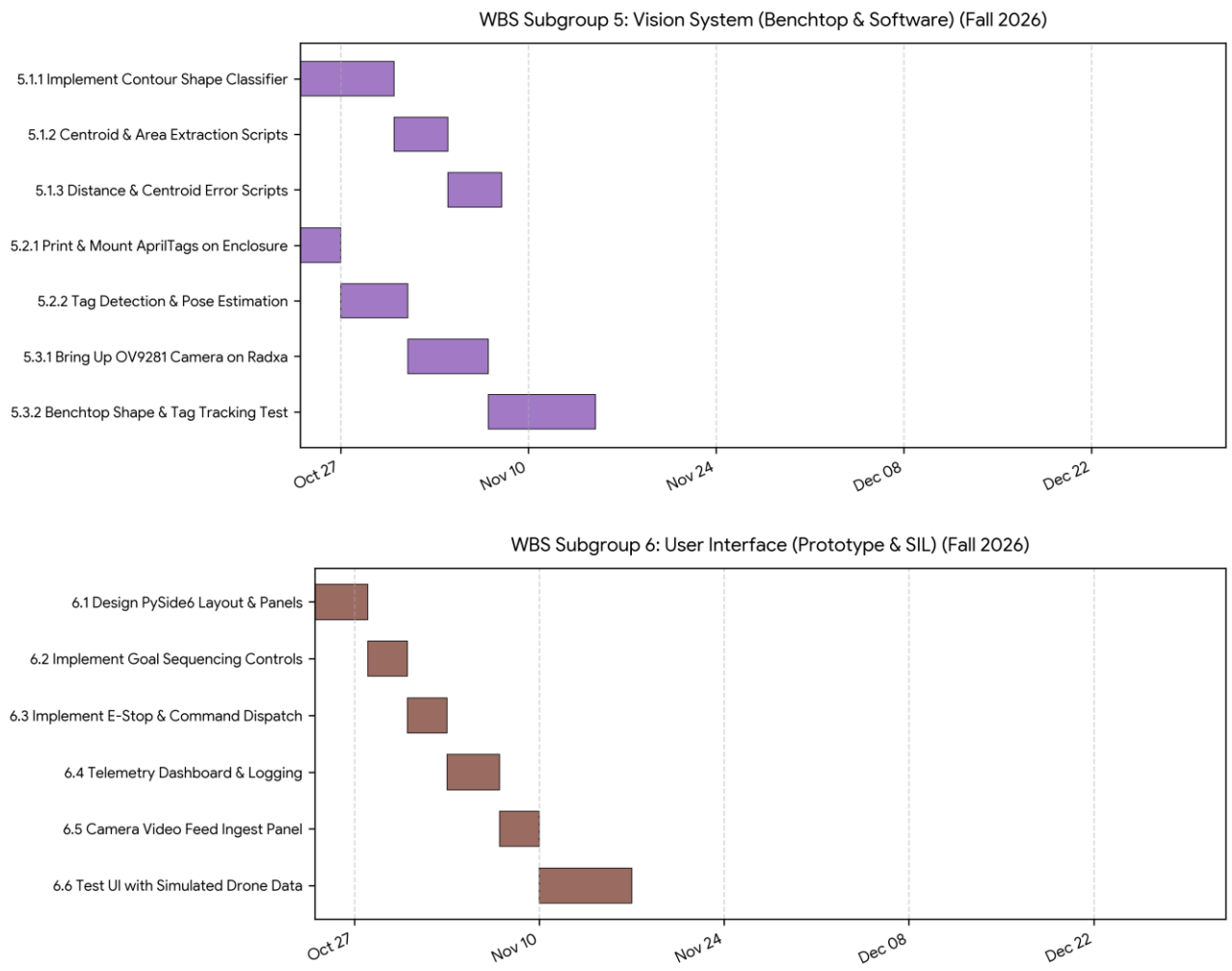
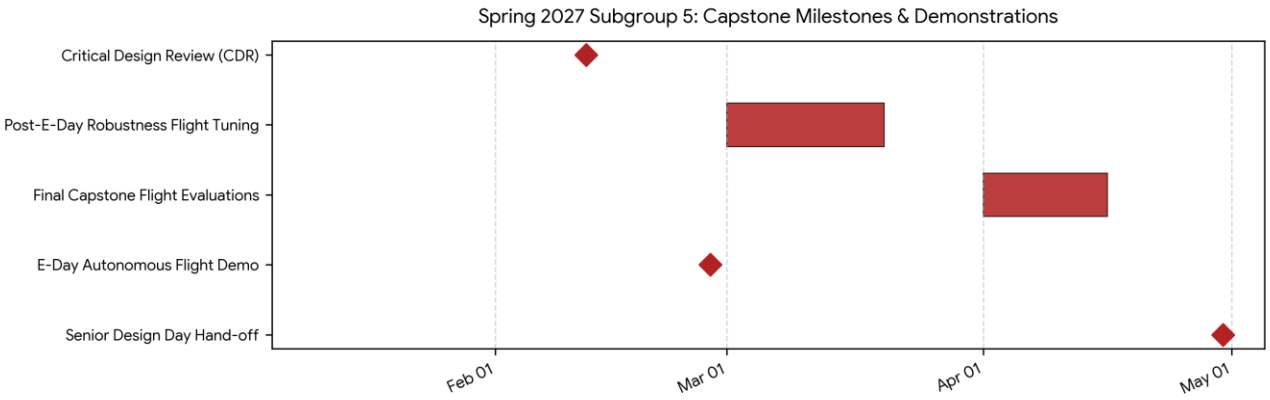
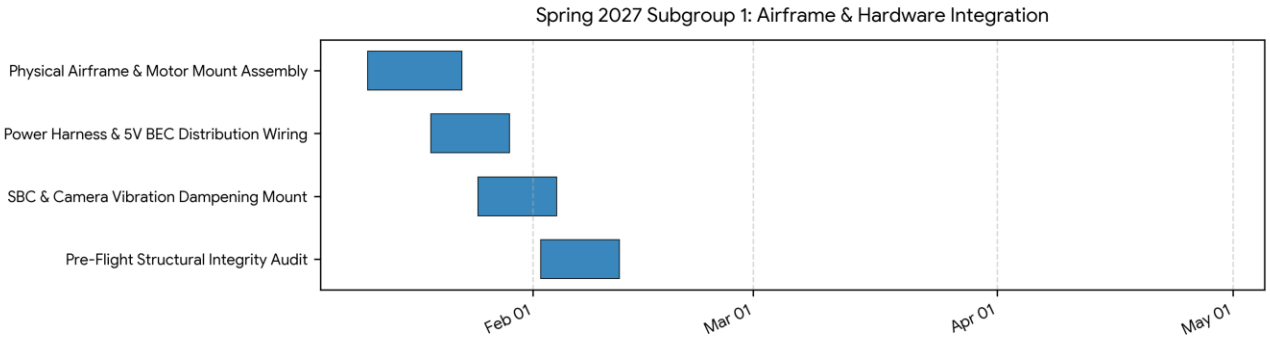


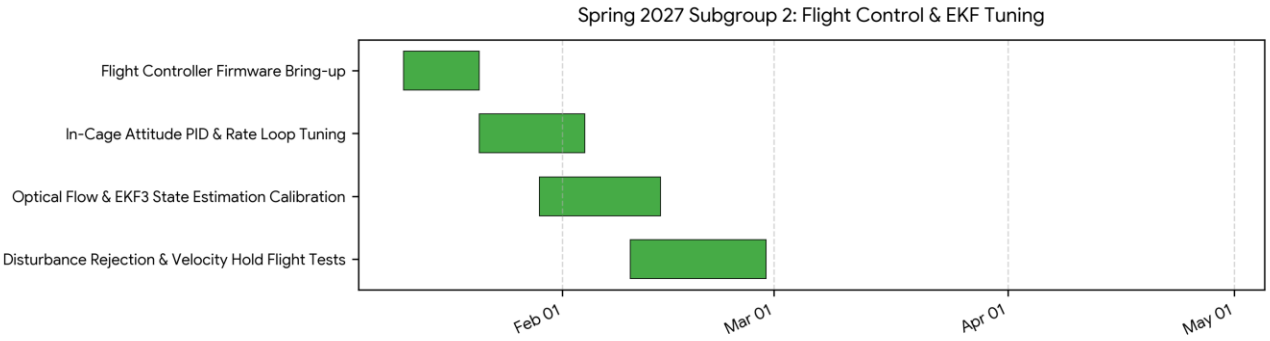

Figure 16. Gantt Charts Outlining WBS Fall 2026

To schedule development progression through the second semester leading up to the E-Day showcase and final project evaluation, the team established a dedicated Spring 2027 execution roadmap. Bounded between the resumption of the capstone sequence (January 11, 2027) and Senior Design Day (April 30, 2027), the spring schedule transitions from modular subsystem verification to unified autonomous operation within the netted flight enclosure.

As illustrated in Figure 17, early spring efforts focus on full system integration: the Radxa Zero 3W companion computer and OV9281 global-shutter camera are physically installed onto the airframe, the 5V BEC power harness is finalized, and closed-loop MAVLink communication between the flight controller and autonomy daemons is established. In-cage flight trials throughout late January and February focus on fine-tuning optical flow and EKF3 odometry drift, refining the contour-based shape recognition pipeline, and validating Artificial Potential Field (APF) trajectory navigation. These technical tracks culminate in the Critical Design Review (February 12, 2027) and the live autonomous flight demonstration at University of Kentucky E-Day on February 27, 2027, followed by post-E-Day robustness tuning, final capstone flight trials, and the Senior Design Day showcase on April 30, 2027.


Spring 2027 Subgroup 5: Capstone Milestones & Demonstrations
Critical Design Review (CDR)
Post-E-Day Robustness Flight Tuning
Final Capstone Flight Evaluations
E-Day Autonomous Flight Demo
Senior Design Day Hand-off
Feb 01
Mar 01
Apr 01
May 01


Spring 2027 Subgroup 1: Airframe & Hardware Integration
Physical Airframe & Motor Mount Assembly
Power Harness & 5V BEC Distribution Wiring
SBC & Camera Vibration Dampening Mount
Pre-Flight Structural Integrity Audit
Feb 01
Mar 01
Apr 01
May 01


Spring 2027 Subgroup 2: Flight Control & EKF Tuning
Flight Controller Firmware Bring-up
In-Cage Attitude PID & Rate Loop Tuning
Optical Flow & EKF3 State Estimation Calibration
Disturbance Rejection & Velocity Hold Flight Tests
Feb 01
Mar 01
Apr 01
May 01

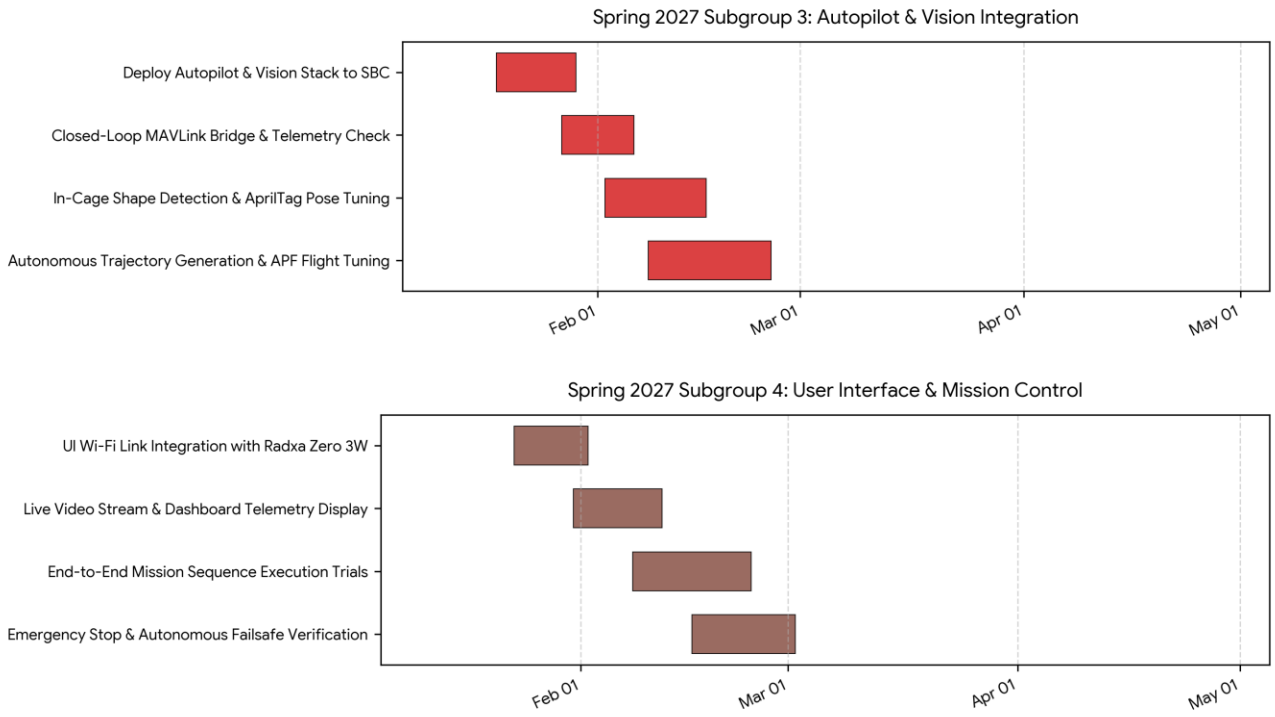



Figure 17. Gantt Charts Outlining Subsystem Integration Spring 2027

## *6.4 Costs*

The budget for this project is $500 and is provided by our project sponsor, Dr. Robert Adams. To accurately track spending, two separate financial breakdowns were created: a development cost breakdown and a rebuild cost breakdown. The development cost breakdown details all financial estimations for the actual development of our drone. These estimations are based on testing materials, spare parts, and actual purchases this semester for each subsystem. The rebuild cost breakdown lists only the specific parts needed to build the final drone and its accessories (net enclosure). This is a projected estimate as the project is still in development. It additionally includes items already in possession from previous teams. To account for uncertainty, minimum $cost_a$, maximum $cost_b$, and most likely $cost_m$, including possible tax and shipping costs, were identified for each component and the equation used is below. The resulting financial breakdowns are provided below in Tables 10 and 11.

$$cost = \frac{cost_a + 4cost_m + cost_b}{6} \tag{8}$$

Table 9. Development Cost Breakdown

| Development Cost Breakdown | | | | |
|---|---|---|---|---|
| **Item Name** | **Cost(a)** | **Cost(m)** | **Cost(b)** | **Calculated Cost** |
| Flywoo 5V/12V 2A BEC module V 2.0 | $6.99 | $11.99 | $14.49 | $11.57 |
| Radxa ZERO 3W Tiny RK3566 SBC | $50.99 | $59.99 | $74.98 | $60.99 |
| GR2004 Motors | $65.96 | $81.44 | $103.96 | $82.61 |
| Storm2.0 4S 850mAh 150C LiPo Battery | $36.98 | $53.43 | $67.98 | $53.11 |
| CLRacing Race LED/PCB Wire 4 Pack - 8x25mm | $4.99 | $9.98 | $11.99 | $9.48 |
| Reflective tape | $12.11 | $18.06 | $21.06 | $17.57 |
| 3D Printer Filament | $31.90 | $36.99 | $40.99 | $36.81 |
| OV9281 Global Shutter Camera Module for Raspberry Pi 4B / 5 (1MP, FOV 79°) | $29.99 | $35.90 | $39.99 | $35.60 |
| **Total Cost Calculations:** | **$239.91** | **$307.78** | **$375.44** | **307.745** |

Table 10. Rebuild Cost Breakdown

| Rebuild Cost Breakdown | | | | |
|---|---|---|---|---|
| **Item Name** | **Cost(a)** | **Cost(m)** | **Cost(b)** | **Calculated Cost** |
| Flywoo 5V/12V 2A BEC module V 2.0 | $6.99 | $11.99 | $14.49 | $11.57 |
| Radxa ZERO 3W Tiny RK3566 SBC | $50.99 | $59.99 | $74.98 | $60.99 |
| GR2004 Motors | $65.96 | $81.44 | $103.96 | $82.61 |
| Storm2.0 4S 850mAh 150C LiPo Battery | $36.98 | $53.43 | $67.98 | $53.11 |
| CLRacing Race LED/PCB Wire 4 Pack - 8x25mm | $4.99 | $9.98 | $11.99 | $9.48 |
| Reflective tape | $12.11 | $18.06 | $21.06 | $17.57 |
| 3D Printer Filament | $31.90 | $36.99 | $40.99 | $36.81 |
| OV9281 Global Shutter Camera Module for Raspberry Pi 4B / 5 (1MP, FOV 79°) | $29.99 | $35.90 | $39.99 | $35.60 |
| PVC Pipe | $64.99 | $74.99 | $80.99 | $74.32 |
| Netting | $13.99 | $16.99 | $19.99 | $16.99 |
| Gates | $3.99 | $4.99 | $7.99 | $5.32 |
| Flywoo Goku GN745 | $105.99 | $126.99 | $138.99 | $125.49 |
| Micoair MTF-02P - Optical Flow Sensor | $26.99 | $33.99 | $35.99 | $33.16 |
| VL53L0X - Time of Flight Sensor | $15.99 | $19.88 | $25.88 | $20.23 |
| **Total Cost Calculations:** | **$471.85** | **$585.61** | **$685.27** | **$583.26** |

# 7 Conclusion and Outlook on Future Work

At this stage, the project team has completed the preliminary design process, including subsystem identification, hardware selection, and evaluation of the previous year's prototype. Reusable components, including the test enclosure, flight controller, and sensors, have been retained, while upgrades to the single-board computer, camera system, motors, and airframe design are expected to improve processing capability, reliability, and flight performance. Initial research has been conducted on autonomous flight control, sensor integration, and regulatory requirements to ensure the proposed system can operate safely and effectively in an indoor environment. Moving forward, the team will focus on assembling and integrating the selected hardware, configuring the flight controller and sensor systems, and validating subsystem functionality through bench testing. Future work will include calibration of the flight stabilization and position estimation systems, implementation of autonomous control algorithms, and extensive flight testing to verify stable hovering, navigation, and obstacle detection capabilities. Successful completion of these tasks will allow the team to demonstrate a fully functional autonomous indoor drone that meets the project's engineering requirements and performance objectives.

# 8 References

[1] DJI Education, "Tello EDU Brochure," Jul. 2019. [Online]. Available: https://ired.co.uk/wp-content/uploads/2019/07/DJI-Tello-EDU-Brochure-iRed.pdf. [Accessed: Oct. 22, 2026].

[2] Amazon, "Prime Air Drone Delivery." [Online]. Available: https://www.amazon.com/Prime-Air-Drone-Delivery/b?ie=UTF8&node=206533607011. [Accessed: Oct. 23, 2026].

[3] Electronic Code of Federal Regulations, “14 CFR Part 107—Small Unmanned Aircraft Systems.” [Online]. Available: https://www.ecfr.gov/current/title-14/chapter-I/subchapter-F/part-107

[4] ASTM International, Standard Practice for Operational Risk Assessment of Unmanned Aircraft Systems (UAS), ASTM F3178-24, 2024. [Online]. Available: https://store.astm.org/f3178-24.html

[5] ASTM International, Standard Specification for Design and Construction of a Small Unmanned Aircraft System (sUAS), ASTM F2910-22, 2022. [Online]. Available: https://store.astm.org/f2910-22.html

[6] ASTM International, Standard Specification for Design of the Command and Control System for Small Unmanned Aircraft Systems (sUAS), ASTM F3002-22, 2022. [Online]. Available: https://store.astm.org/f3002-22.html

[7] International Electrotechnical Commission, Secondary Cells and Batteries Containing Alkaline or Other Non-Acid Electrolytes—Safety Requirements for Portable Sealed Secondary Cells, and for Batteries Made From Them, for Use in Portable Applications—Part 2: Lithium Systems, IEC 62133-2:2017+A1:2021, 2021. [Online]. Available: https://webstore.iec.ch/en/publication/70017

[8] Overview of materials for polylactic acid (PLA) biopolymer, [Online]. Available: https://www.matweb.com/search/datasheet.aspx?matguid=ab96a4c0655c4018a8785ac4031b9278&n=1&ckck=1 [accessed Oct. 23, 2026].

[9] Mitsubishi carbon-P pet-G, 15% carbon fiber filled 3D printing filament, [Online]. Available: https://matweb.com/search/datasheettext.aspx?matguid=a26c338abdda4bdeb0c2185c7f322d03 [accessed Oct. 23, 2026].

[10] LIG Power, "Drone Motor Specifications and Parameters," LIG Power Blog. [Online]. Available: https://www.ligpower.com/blog/drone-motor-specifications-and-parameters.html. [Accessed: Oct. 23, 2026].

[11] Wildair, "Drone Motors: Selection, Performance, and Optimization," Wildair Blog. [Online]. Available: https://blog.wildair.ch/drone-motors/. [Accessed: Oct. 23, 2026].

[12] Skyvolt, "FPV Drone Battery Guide," Skyvolt Technical Insights. [Online]. Available: https://www.skyvolt-tech.com/en/blogs/technical-insights/fpv-drone-battery-guide. [Accessed: Oct. 23, 2026].

[13] Teledyne Vision Solutions, "Rolling vs Global Shutter," *Teledyne Vision Solutions*, 2026. [Online]. Available: https://www.teledynevisionsolutions.com/learn/learning-center/imaging-fundamentals/rolling-vs-global-shutter/. [Accessed: Oct. 23, 2026].

[14] OpenCV, "OpenCV Library" (Version 4.x.x). [Online]. Available: https://opencv.org. [Accessed: Oct. 23, 2026].

# 9 Appendix

## *9.1 Functional Decompositions*

### *9.1.1 Level 0 – Device Overview*

The Level 0 functional decomposition represents the complete autonomous drone system as a single black-box entity interacting with its external environment and user base. As illustrated in Figure 18, the system accepts two primary external inputs: the operator-supplied user input defining the target gate sequence, and power from charger delivered via the swappable onboard battery pack. In response, the system produces two primary outputs: the physical course traversal demonstration through the netted indoor arena, and live flight logs streamed wirelessly for real-time monitoring.

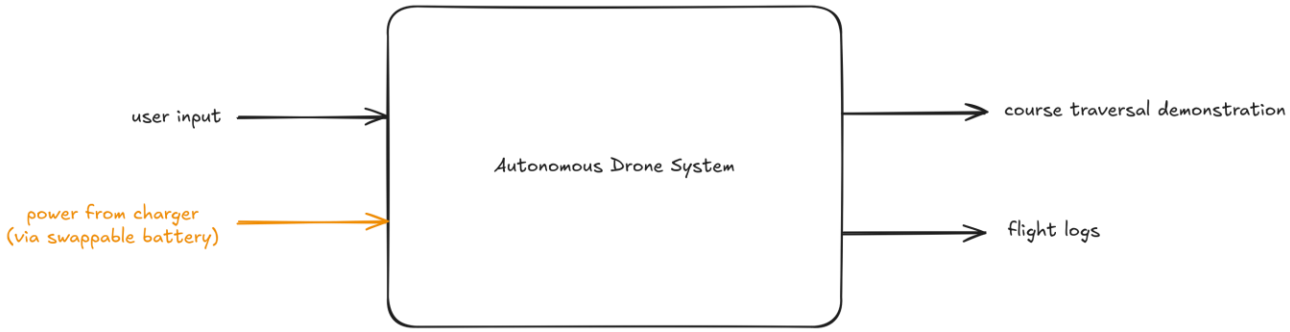



Figure 18. Level 0 Functional Decomposition

### *9.1.2 Level 1 – Subsystem Integration*

The Level 1 functional decomposition partitions the autonomous drone into five core internal subsystems operating across defined physical and logical boundaries. The User Interface subsystem ingests the operator's mission commands and communicates wirelessly via Wi-Fi with the Autopilot module, transmitting sequence data while receiving telemetry and logs. The Power subsystem distributes regulated voltage from the battery pack to all active electronic modules while supplying battery status datagrams to the flight controller. The Flight subsystem, comprising the STM32F745 flight controller and onboard inertial and optical sensors, interfaces with the Autopilot module via high-speed UART MAVLink to exchange flight commands and sensor data. The Single-Board Computer houses the Autopilot and Vision subsystems, where the Vision module captures camera frames and passes processed 3D target coordinates to the Autopilot via a local Unix socket. Finally, the Chassis and Propulsion subsystem provides the physical unibody frame, motor mounts, and Ducted propeller assembly required to execute the autonomous course traversal.

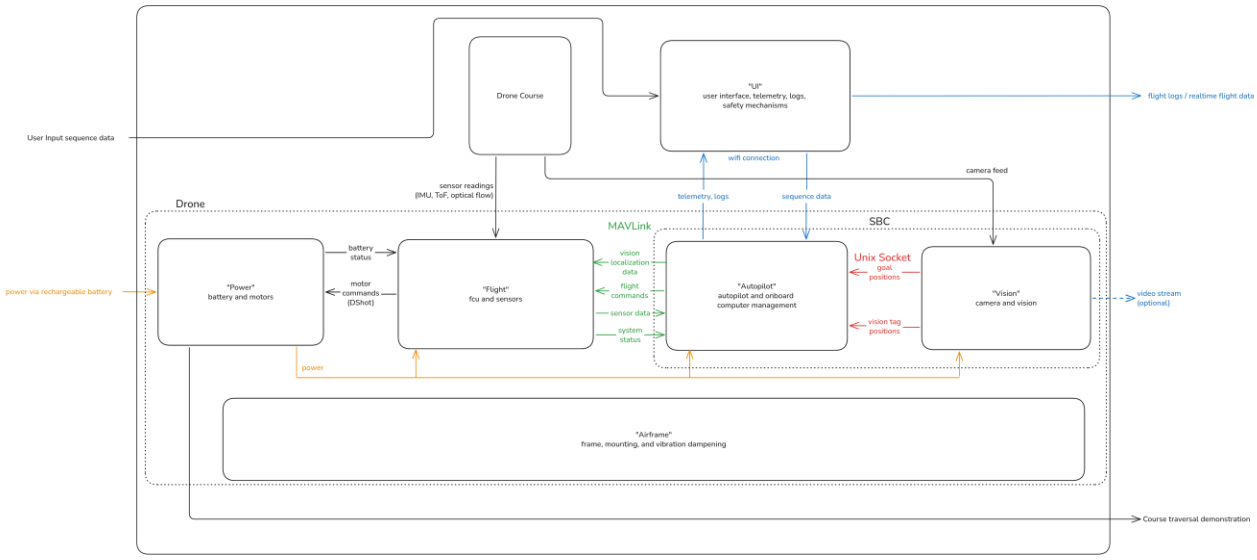



Figure 19. Level 1 Functional Decomposition

### *9.1.3 Level 2 – Airframe*

The airframe is connected to all components, and thus a level 2 functional decomposition has not been included for clarity.

### *9.1.4 Level 2 – Power System*

This Level 2 functional decomposition illustrates the inner workings of the Power System, including the battery and motors, as well as the stepped down 5V created by the BEC for components which require less input voltage. The IO table provides additional clarification on the source, destination, and purpose of system inputs and outputs.

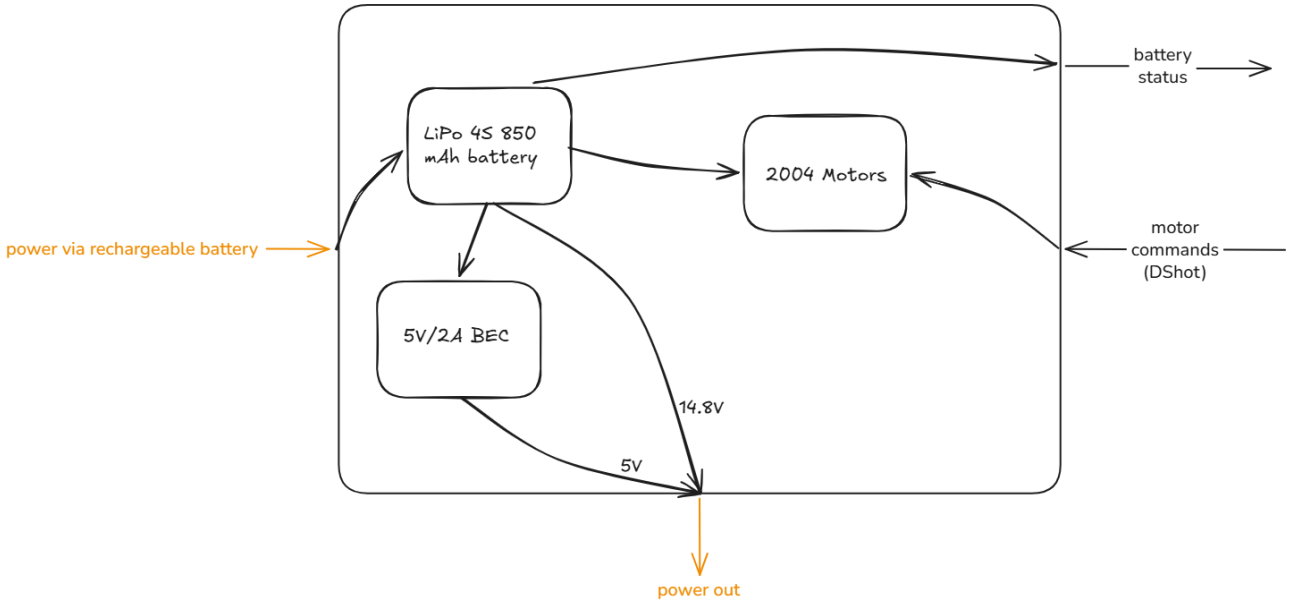



Figure 20. Power System Level 2 Functional Decomposition

Table 11. Power System IO Table

| **Interface** | **Direction** | **Source/ Destination** | **Description** |
|---|---|---|---|
| Power via rechargeable battery | Input | External | External power input in form of LiPo battery |
| Motor commands | Input | “Flight” | Flight controller provides modulated signals to control motor function |
| Power out | Output | “Flight”, “Autopilot”, “Vision” | Power is distributed from 5V and 14.8V lines to support needed inputs to other subsystems |
| Battery status | Output | “Flight” | Critical link which relays battery health to flight controller |

### *9.1.5 Level 2 – Sensors*

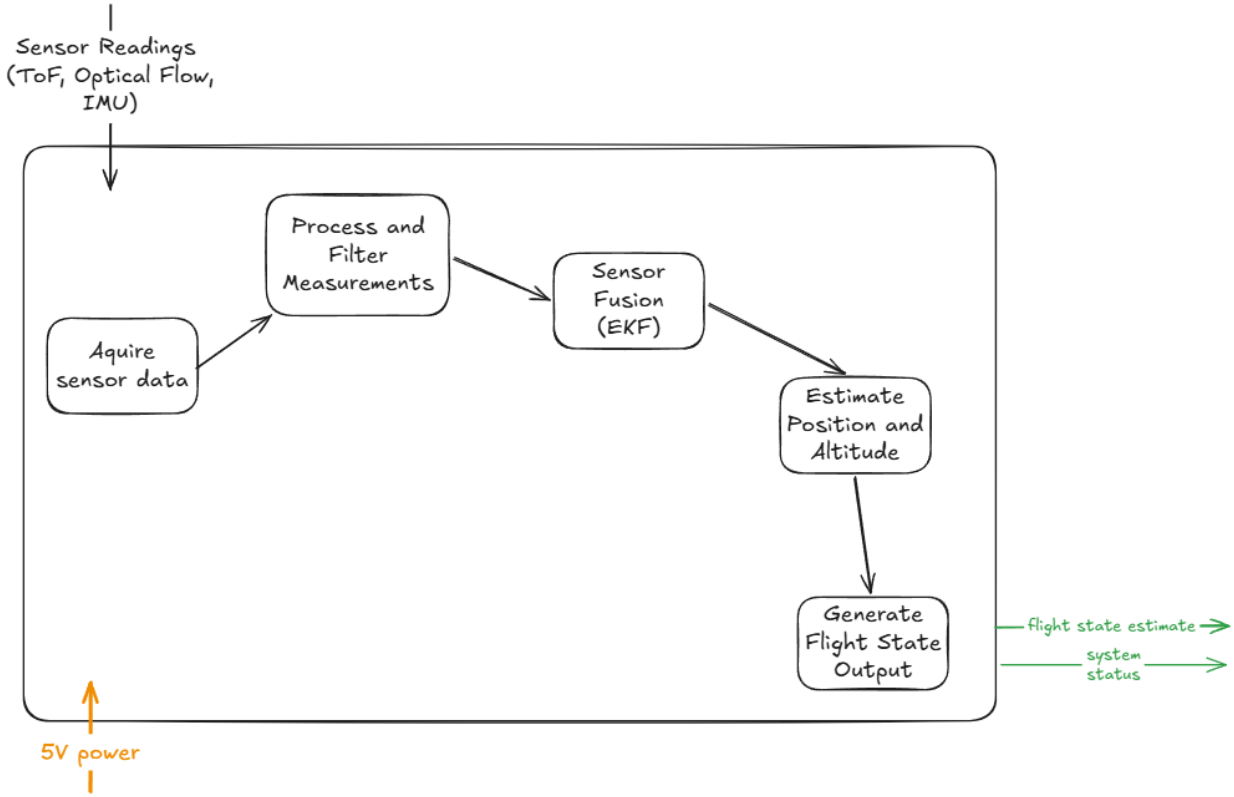



Figure 21. Sensors Level 2 Functional Decomposition

The figure above illustrates the internal data processing functions and operational boundaries of the Sensor subsystem. Operating as the primary source of flight state information for the autonomous drone, this subsystem acquires measurements from the inertial measurement unit (IMU), optical flow sensor, and Time-of-Flight (ToF) sensor and converts them into meaningful position, altitude, and attitude estimates.

Table 12. Sensor IO Table

| **Interface** | **Direction** | **Source/ Destination** | **Description** |
|---|---|---|---|
| Sensor Readings | Input | ToF, Optical Flow, IMU sensors | Acquires measurements from sensors and converts into meaningful drone location estimates. |
| 5V Power | Input | Power Subsystem | Supplies continuous electrical power to the Flywoo GOKU GN745. |
| Flight State Estimate | Output | Autopilot Subsystem | Enable stable hovering, navigation and autonomous operation. |
| System Status | Output | Autopilot Subsystem | Critical link between physical sensors and flight control algorithms. |

### *9.1.6 Level 2 – Autopilot*

The Level 2 functional decomposition in Figure 22 illustrates the internal software modules, data flow paths, and operational boundaries of the Autopilot and Single-Board Computer (SBC) subsystem. Operating as the central decision-making hub of the HEXAGON Flight autonomous drone, this subsystem coordinates high-level mission goals and translates vision estimates into deterministic flight setpoints.

As depicted in Figure 22, the internal software stack is organized into five primary functional blocks:

- State Machine (Master Coordinator): Manages mission lifecycle execution, evaluates system state structures, and sequences operational modes (Takeoff, Goal Seek, Traversal, Land) while handling safety watchdogs and emergency overrides.
- MAVLink Daemon (Flight Interface): Manages bidirectional serial communication with the low-level flight controller over hardware UART (9,216 baud), serializing flight commands and passing through vision localization data while decoding incoming sensor data and system status.
- Vision Reader (IPC Server): Operates a local Unix domain socket server to ingest real-time goal positions and vision tag positions from the companion vision process, packaging them into structured vision data blocks.
- Goal Estimator (State Estimation): Ingests raw frame-by-frame 3D coordinate hypotheses from the Vision Reader, filters spatial noise, and maintains stable arena-frame goal poses passed to the path planner.
- Path Planner (Trajectory Generation): Implements trajectory and waypoint solvers to compute motion setpoints based on current goal poses, feeding path setpoints back into the master State Machine.

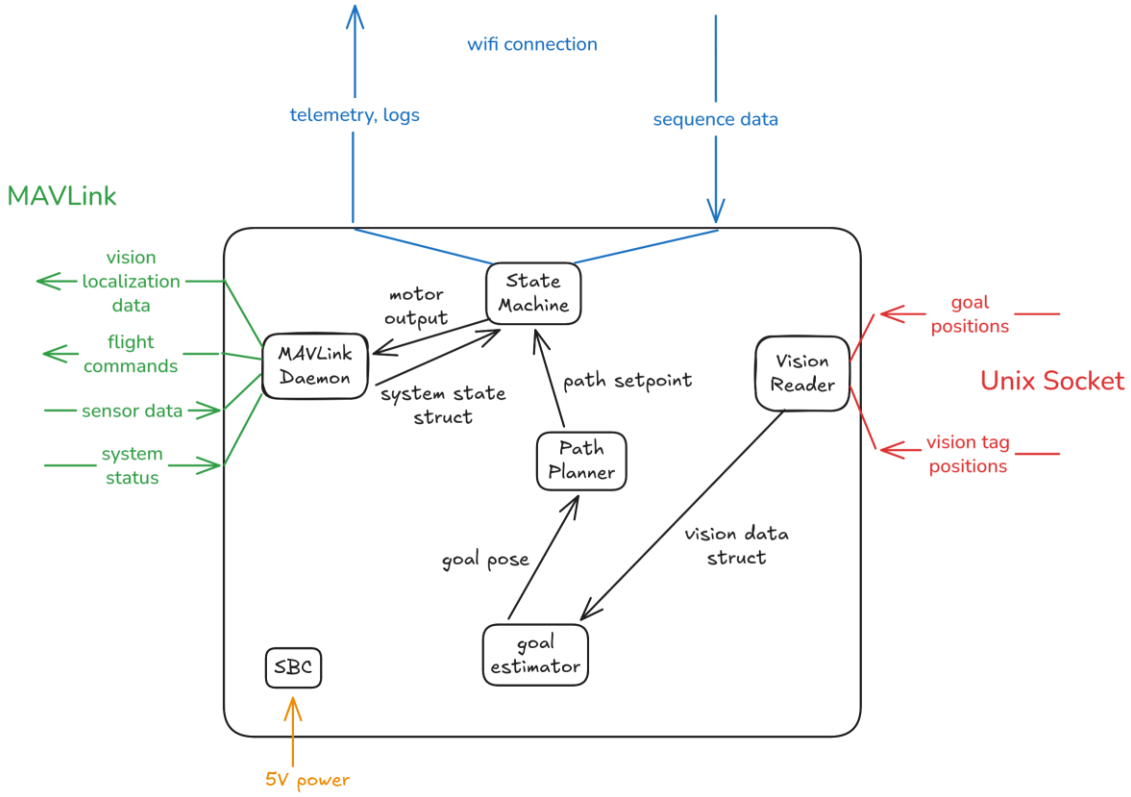



Figure 22. Autopilot Level 2 Functional Decomposition

The functional inputs and outputs governing the Autopilot subsystem across its modular boundaries correspond directly to the MAVLink, Unix Socket, WiFi, and Power interfaces:

Table 13. Autopilot IO Table

| **Interface** | **Direction** | **Source / Destination** | **Description** |
|---|---|---|---|
| UI Sequence Data | Input | UI Subsystem (via WiFi) | Ingests operator-selected geometric target gate sequences and configuration parameters. |
| Vision Goal / Tag Stream | Input | Vision Subsystem (via IPC) | Receives real-time 3D target coordinates (goal positions) and AprilTag vectors (vision tag positions). |
| FCU Sensor & Status | Input | Flight Subsystem (via MAVLink via UART) | Ingests raw sensor data (IMU, optical flow, lidar) and system status flags. |
| SBC Regulated Power | Input | Power Subsystem (5V BEC) | Supplies continuous electrical power to the Radxa Zero 3W single-board computer. |
| Flight Commands Setpoints | Output | Flight Subsystem | Transmits high-level velocity and position setpoints (flight commands) to the ArduPilot flight controller. |
| Vision Localization Data | Output | Flight Subsystem | Forwards processed vision position estimates (vision localization data) to the FCU's EKF3 estimator to zero drift. |
| UI Telemetry & Logs | Output | UI Subsystem | Streams live vehicle telemetry, system logs, and state indicators back to the operator dashboard. |

### *9.1.7 Level 2 – Vision System*

The vision system level 2 functional decomposition is shown below in Figure 23 along with the IO table for the vision system in Table 14. As the eyes of the drone, the vision system turns raw image frames from the camera into useful positional data for the drone and its target gates that are sent to the autopilot subsystem existing on the same single board computer.

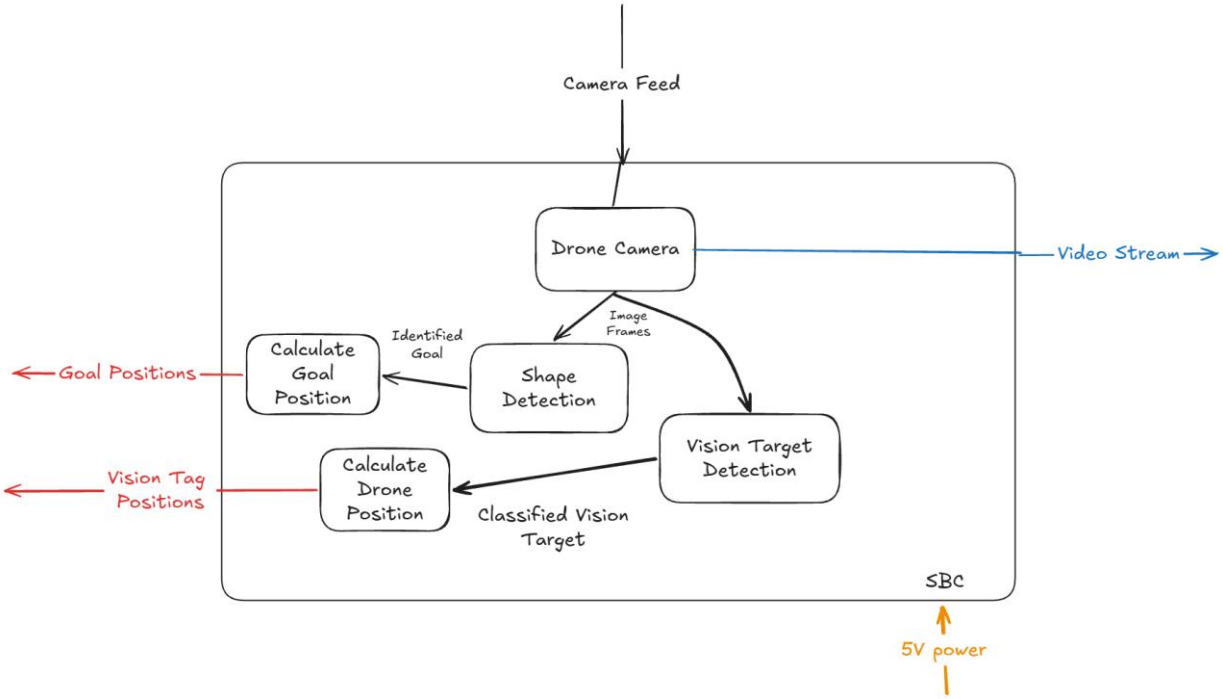



Figure 23. Vision System Level 2 Functional Decomposition

Table 14. Vision System IO Table

| **Interface** | **Direction** | **Source / Destination** | **Description** |
|---|---|---|---|
| Camera Feed | Input | Drone Course Subsystem | The drone camera captures raw images of the drone enclosure and shaped gates. |
| 5V Power | Input | Power Subsystem | Supplies continuous electrical power to the Radxa Zero 3W single-board computer. |
| Video Stream | Output | UI Subsystem | Sends a live video stream from the drone's camera to be viewed on the UI. |
| Goal Positions | Output | Autopilot Subsystem | Sends the estimated goal positions to the autopilot software to generate flight navigation. |
| Vision Tag Positions | Output | Autopilot Subsystem | Sends the estimated vision tag positions to the autopilot software to generate flight navigation. |

### *9.1.8 Level 2 – User Interface*

The level 2 functional decomposition for the User Interface is shown in Figure 24, along with the UI input/output (IO) table in Table 15. The UI provides the user with an interface to select the goal sequence, monitor mission status, and initiate an emergency stop. It communicates with the onboard single-board computer (SBC) through a wireless Wi-Fi connection to send goal sequences, receive acknowledgments, and display telemetry and status updates. The UI also provides a live camera feed and mission controls to support the autonomous drone demonstration.

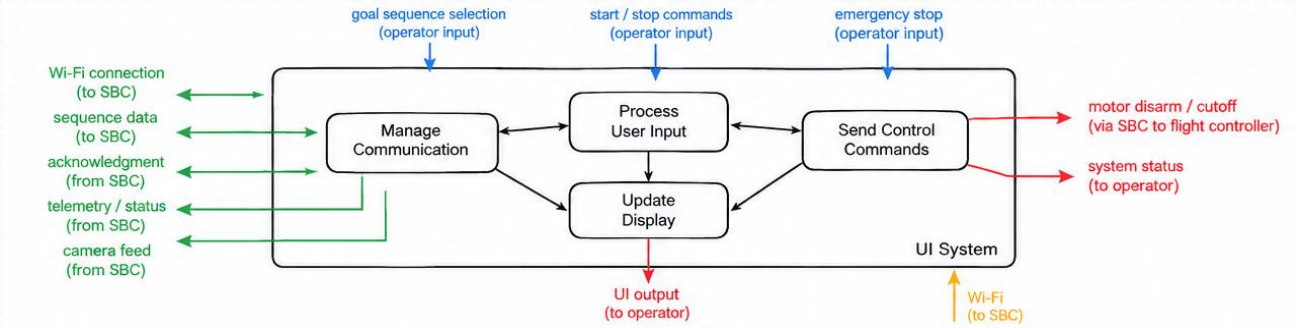



Figure 24. User Interface Level 2 Functional Decomposition

Table 15. User Interface IO Table

| **Interface** | **Direction** | **Source / Destination** | **Description** |
|---|---|---|---|
| User Input (Goal Sequence) | Input | Operator → UI | Allows the operator to select the desired order of the circle, triangle, and square. |
| Mission Control (Start/Stop) | Input | Operator → UI | Allows the operator to start a mission or activate the Emergency Stop. |
| UI Commands (Sequence/Stop) | Output | UI → SBC | Sends the selected goal sequence or stop command to the SBC through Wi-Fi. Sends a live video stream from the drone's camera to be viewed on the UI. |
| Acknowledgment | Input | SBC →UI | Confirms that the SBC received the goal sequence or provides a response to a control command. |
| Telemetry / Status Data | Input | SBC →UI | Provides drone status and mission progress information for display. |
| Camera Feed | Input | SBC →UI | Provides the live video feed from the drone's camera for display in the UI. |
| UI Outputs (Display/Alerts) | Output | UI → Operator | Displays video, telemetry, mission progress, connection status, and alerts. |
| Emergency Stop Response | Output | UI → Flight Controller (through SBC) | Sends a stop command through the SBC to initiate motor disarming and output cutoff. |
| System Readiness Status | Output | UI → Operator | Indicates communication status and system readiness, including following a battery swap. |

### *9.1.9 Level 2 – Drone Course*

The level 2 functional decomposition of the drone course shows the relationship between the enclosure and the data it provides to the drone. Once in the course, the drone takes in sensor readings from within the enclosure, and then the camera identifies any goals and vision targets that are present. The outputs are further described in Table 16.

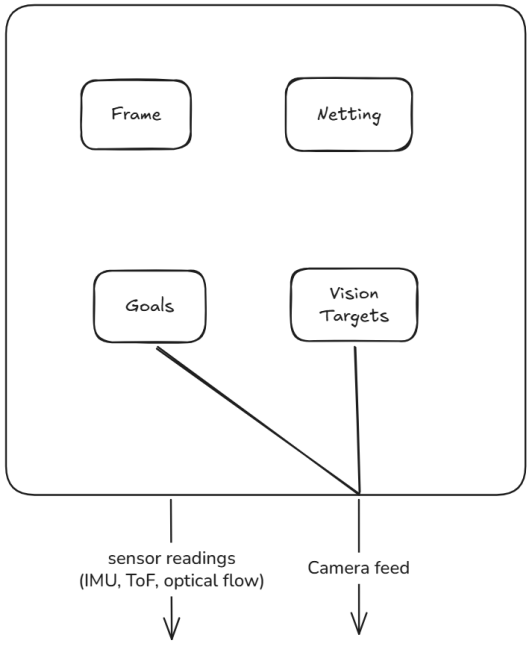



Figure 25. Drone Course Level 2 Functional Decomposition

Table 16. Drone Course IO Table

| **Interface** | **Direction** | **Source / Destination** | **Description** |
|---|---|---|---|
| Sensor Readings | Output | Flight control | Provides the necessary information for the ESC/Flight controller to execute controlled flight commands. |
| Camera Feed | Output | Vision system | Provides the necessary targets for the vision system to operate. |